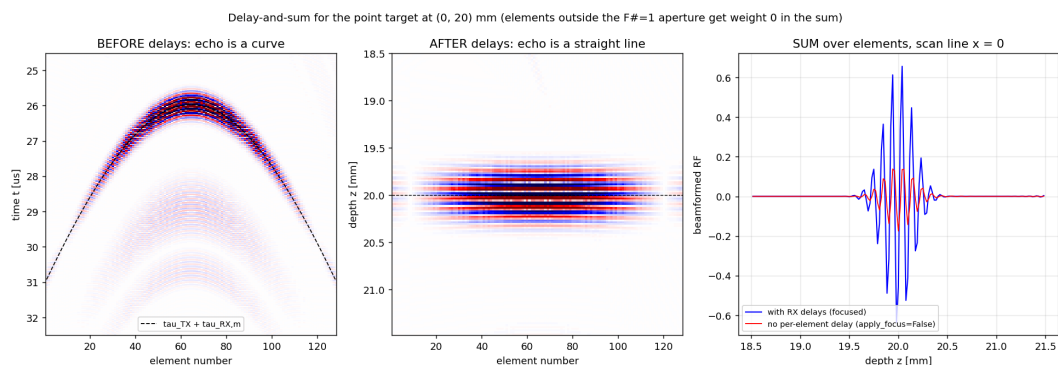


Team 5: Receive (RX) Beamforming

Delay-and-sum: turning 128 channels of echoes into one image line per scan line

NITK-UsoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- Input: raw RF from all 128 receive elements of one broadcast transmit (Philips L12-4 model, 8 MHz, fs 40 MHz).
- Method: delay-and-sum with dynamic receive focus, Hann apodization and F-number 1.0 (das_beamform).
- Output: beamformed RF image, 3118 depths x 256 scan lines (0-60 mm, x = -10..+10 mm).
- Team 5's verification tests: 4/4 pass (depth errors 0.008 to 0.040 mm).
- One command: `python3 run_team5.py` (all outputs go to output/).



The core idea of this team: echoes from one point arrive as a curve across the elements (left); after the right delays they line up (middle) and add up to a strong, sharp peak (right).

Contents

1 Overview	3
Who wrote what	3
Folder contents	3
2 Theory	4
2.1 The probe and the numbers used	4
2.2 Why the echo of one point is a curve	4
2.3 The delays	4
2.4 Dynamic receive focusing	5
2.5 Apodization (receive window)	5
2.6 F-number and the growing aperture	5
2.7 Axial and lateral resolution, side lobes and grating lobes	5
2.8 The time-axis rule: always np.interp on the real t_axis	6
3 Code explanation	7
3.1 receive_beamforming.py: das_beamform()	7
3.2 verify_beamformer.py: Team 5's tests	7
3.3 mock_data.py: test RF for the tests	8
3.4 Files added for this package	8
3.5 Support files (short description only)	9
3.6 Full listings	9
4 Input	22
5 Output	23
5.1 Verification tests	23
5.2 The delay idea	23
5.3 Beamformed RF and display images	24
5.4 Measured positions and resolution	27
5.5 Apodization and aperture comparison	28
5.6 Full console output	29
5.7 List of output files	32
6 How to run	33
Troubleshooting	33
7 Check yourself	34

1 Overview

What Team 5 does and what is in this folder

An ultrasound probe sends a short pulse into the body and then listens with all of its elements. Every element records its own signal (a **channel**). An echo from one small point reaches each element at a slightly different time, because the distances are different. **Receive beamforming** uses the geometry to undo these time differences and adds the channels together, so that echoes from the point we look at add up strongly and echoes from other places do not. Team 5 does this with the classic **delay-and-sum (DAS)** method, for every depth of every scan line.

Place in the pipeline: Team 5 takes the raw per-element RF produced by the propagation and tissue stages (Teams 3 and 4) and gives a beamformed RF image to the next stage, which does envelope detection, log compression and display.

Who wrote what

File	Origin	Role
code/verify_beamformer.py	Team 5 (delivered by the team)	Team 5's four verification tests: focusing, correct vs wrong delay, lateral and depth localisation.
code/receive_beamforming.py	Written during project integration, verified with Team 5's tests	das_beamform(): the delay-and-sum beamformer. Team 5's test file was used as its specification.
code/mock_data.py	Written during project integration	Makes point-target test RF for the tests (calls simulate_rf from support/).
code/config.py	Added for this package	Local copy of the project's simulator/config.py: same numbers and functions, only the folder paths point inside this package.
code/support/*	Copied unchanged from other teams (NOT Team 5's work)	Input generator: Team 3's nitk_usoundsim package, Team 4's tissue_interaction.py and phantom.py, and the integration module tissue_phantoms.py.
run_team5.py	Added for this package	Runs everything with one command and saves all outputs.

The three Team 5 files are byte-identical copies of the project files (checked with cmp). They are not edited; the runner calls them.

Folder contents

```

1 Team_5_RX_Beamforming/
2   README.md                how to run (short)
3   requirements.txt         numpy, scipy, matplotlib
4   run_team5.py             ONE command: tests + beamforming + plots + measurements
5   Team_5_RX_Beamforming_Explanation.pdf  this document
6   code/
7       receive_beamforming.py  das_beamform() (delay-and-sum)
8       verify_beamformer.py    Team 5's 4 verification tests
9       mock_data.py           point-target test RF for the tests
10      config.py              probe / axes / receive settings (local copy)
11      support/               input generator from other teams (not Team 5's work):
12                             nitk_usoundsim/ (Team 3), tissue_interaction.py and
13                             phantom.py (Team 4), tissue_phantoms.py, a note file
14   input/                   point_targets_rf.npz, cyst_rf.npz (raw RF)
15   output/                  everything made by run_team5.py

```

2 Theory

The ideas behind delay-and-sum, in easy words

2.1 The probe and the numbers used

Quantity	Value	Meaning
Elements	128	small transducers side by side along x, at depth z = 0
Pitch	0.30 mm	distance between element centres (aperture 38.4 mm)
Centre frequency f0	8 MHz	Philips L12-4 reference model
Sampling frequency fs	40 MHz	one RF sample every 25 ns
Speed of sound c	1540 m/s	assumed the same everywhere (soft tissue)
Wavelength	$c / f_0 = 0.1925$ mm	the pitch is 1.56 wavelengths
Scan lines	256, x = -10 .. +10 mm	line spacing 0.078 mm
Depth axis	0 .. 60 mm, step $c/(2 f_s) = 0.01925$ mm	3118 depth samples
Receive window / F-number	Hann, F# = 1.0	config.RX_APODIZATION, config.RX_F_NUMBER

2.2 Why the echo of one point is a curve

In this simulator all 128 elements fire at the same moment (a **broadcast** transmit, no transmit focusing). The wave reaches a point (x, z) first from the nearest element. After reflection, the echo travels back to every element m. The element right above the point is closest, so it hears the echo first; elements further to the side hear it later. Plotted as element number against time, the echo is a curve (close to a hyperbola). Example from the run: for the point at (0, 20) mm the centre element hears the echo at 25.975 us and the edge element at 30.923 us, which is 4.95 us later (about 198 samples).

2.3 The delays

For scan line x_l and depth z, the time for the transmit wave to reach the point is taken from the nearest element n:

$$\tau_{TX}(z) = \min_n \frac{\sqrt{(x_l - x_n)^2 + z^2}}{c}$$

and the time for the echo to go back to receive element m is

$$\tau_{RX,m}(z) = \frac{\sqrt{(x_l - x_m)^2 + z^2}}{c}$$

So element m must be read at time $t_m(z) = \tau_{TX}(z) + \tau_{RX,m}(z)$. Delay-and-sum then adds the (weighted) samples:

$$s_l(z) = \frac{\sum_m w_m(z) rf_m(\tau_{TX}(z) + \tau_{RX,m}(z))}{\sum_m w_m(z)}$$

If the point really is at (x_p, z) , all channels are read exactly at the echo, the signals are in phase and they add up (constructive sum). For any other place, the samples have different phases and mostly cancel. Dividing by the sum of the weights keeps a point target at the same brightness at every depth.

Key idea

Delay each channel by its own travel time, then add. Points that are in focus add up in phase; everything else mostly cancels. Do this for every depth of every scan line.

2.4 Dynamic receive focusing

The delays are recomputed for **every depth** z , not only for one focal depth. This is called dynamic receive focusing: the image is in focus at all depths. Team 5's test 2 checks this: with `apply_focus=False` every element gets the same delay z/c and the peak is much weaker.

2.5 Apodization (receive window)

A sharp edge in the aperture (all weights equal, 'rect') makes strong side lobes: bright copies of a point beside the real point. A smooth window gives the edge elements less weight and lowers the side lobes, but makes the main lobe a little wider. With $u = |x_m - x_l| / a(z)$ (0 at the centre, 1 at the edge of the aperture), the code uses:

$$w_{hann}(u) = \cos^2(\pi u/2), \quad w_{hamming}(u) = 0.54 + 0.46\cos(\pi u), \quad w_{rect}(u) = 1$$

2.6 F-number and the growing aperture

The F-number is depth divided by aperture width. With $F\#$ fixed, the receive aperture grows with depth: near the probe only a few elements are used, deeper down more. The code uses the half-aperture

$$a(z) = \max\left(\frac{z}{2F\#}, 2 \cdot \text{pitch}\right)$$

and only elements with $|x_m - x_l| < a(z)$ take part. With $F\# = 1$ the largest receive angle is $\arctan(1/(2F\#)) = 26.6$ degrees.

2.7 Axial and lateral resolution, side lobes and grating lobes

- **Axial resolution** (along depth) depends on the pulse length: a shorter pulse gives a thinner spot.
- **Lateral resolution** (sideways) depends on wavelength and F-number: roughly proportional to $\lambda \cdot F\#$. A larger aperture (smaller $F\#$) gives a narrower spot.
- **Side lobes** are weaker echoes of a point just beside it, caused by the finite aperture; apodization lowers them.
- **Grating lobes** appear when the pitch is larger than half a wavelength. Here the pitch is 0.30 mm = 1.56λ , so a strong copy can appear at the angle $\sin \theta = \lambda / \text{pitch}$, $\theta = 39.9$ degrees. $F\# = 1$ limits the receive angles to 26.6 degrees, well below 39.9 degrees, so the receive side does not look in the grating-lobe directions.

$$\sin \theta_g = \frac{\lambda}{\text{pitch}} = \frac{0.1925}{0.30} = 0.64 \Rightarrow \theta_g = 39.9^\circ \quad \arctan \frac{1}{2F\#} = 26.6^\circ \quad (F\# = 1)$$

2.8 The time-axis rule: always `np.interp` on the real `t_axis`

The RF time axis does **not** start at $t = 0$. It starts at $t = -0.75$ us (minus half of the pulse length, the convention of the propagation code), so sample number k is at time `t_axis[k]`, not k / fs . If the code used `rf[m, round(tau * fs)]`, every time would be off by 0.75 us, which is $0.75 \text{ us} \times 1540 / 2 = 0.58$ mm in depth, and every target would appear too deep. Therefore `das_beamform` reads every value with `np.interp(tau, t_axis, rf[m])`. This also gives linear interpolation between samples, because the needed times almost never fall exactly on a sample.

3 Code explanation

Every Team 5 file, section by section, then the full listings

3.1 receive_beamforming.py: das_beamform()

Written during project integration, verified with Team 5's tests. One function does all the work:

```
1 beamformed, z_axis = das_beamform(rf, element_positions, scanline_positions, config,
2                                   apply_focus=True, t_axis=None,
3                                   apodization=None, f_number='config')
```

1. **Module docstring** (lines 1-77): explains the inputs, the outputs, the delay formulas, the dynamic aperture and the time-axis rule.
2. **Window tables** _WINDOWS and _WINDOW_U: the Hann, Hamming and rect windows, once over all elements (fixed aperture) and once as a function of u in [0, 1) (dynamic aperture).
3. **Input handling**: converts inputs to float arrays; takes t_axis from config if not given; takes the depth axis z_axis = config.get_depth_axis() and c. A 2-D rf (elements x samples) is one broadcast frame used for every line; a 3-D rf gives one frame per line. Shapes are checked and a clear ValueError is raised if they do not match.
4. **Settings**: apodization defaults to config.RX_APODIZATION ('hann'); f_number='config' means config.RX_F_NUMBER (1.0), None means fixed full aperture. The pitch is found from the element positions and the half-aperture $a(z) = \max(z/(2F\#), 2 \text{ pitch})$ is computed for all depths at once.
5. **Loop over scan lines**: for each line x, τ_{TX} is computed for all depths as the smallest element-to-point distance divided by c.
6. **Loop over elements**: the weight w for element m at every depth (zero outside the aperture; elements that are never used are skipped); the receive delay $\tau_{RX,m}$ (or simply z/c when apply_focus=False); then the RF of element m is read at $\tau_{TX} + \tau_{RX,m}$ with np.interp on the real t_axis (0 outside the recorded time), multiplied by w and added to the line.
7. **Normalisation**: with a dynamic aperture the line is divided by the sum of weights at each depth.
8. **Return**: the beamformed RF (depths x lines) and the depth axis. It is still RF (it oscillates); the envelope is not taken here.

Note on comments in the code

The docstrings of receive_beamforming.py and mock_data.py say that t_axis starts at -1.175 us and that the tau*fs mistake would move targets by about 0.90 mm. Those numbers do not match the current settings (they seem to be left over from an earlier setting). With the 8 MHz probe used now, t_axis starts at -0.75 us (printed by run_team5.py) and the shift would be about 0.58 mm. The rule itself (use np.interp on t_axis) is the same, and the code follows it. The files are left unchanged on purpose.

3.2 verify_beamformer.py: Team 5's tests

Delivered by Team 5. It imports config, the test-RF generator mock_data and das_beamform, and runs four tests. Each prints what it measured and [PASS] or [FAIL].

Test	What it does	Pass rule
1 Point-target focusing	One point at (0, 30) mm, one scan line at $x = 0$; finds the depth of the largest <code> beamformed </code> value.	error < 5 depth samples = 0.096 mm
2 Correct vs incorrect delay	Same data, beamformed with <code>apply_focus=True</code> and <code>False</code> .	focused peak > unfocused peak
3 Lateral localisation	Two points (-4, 25) and (5, 35) mm; all 256 lines (the frame is repeated per line); finds the strongest pixel.	within 2 mm (x) and 3 mm (z) of one target
4 Depth localisation	One point on axis at 15, 40 and 55 mm.	error < 0.5 mm at each depth

Function `report()` prints the PASS/FAIL line. The block at the end runs the four tests in order when the file is started directly.

3.3 mock_data.py: test RF for the tests

Written during project integration. It gives `verify_beamformer.py` the two functions it expects:

- `_check_geometry()`: refuses any array other than the project array, so the test cannot silently use a different geometry.
- `generate_point_target_rf(x, z, ...)`: RF of one point target.
- `generate_two_point_targets_rf(targets, ...)`: RF of several point targets.

Both call `simulate_rf()` from `support/tissue_phantoms.py`, which uses the real propagation and tissue code, not a simplified formula. They return `(rf, t_axis)`.

3.4 Files added for this package

code/config.py is the project's `simulator/config.py` with the same values and functions. Only the top part changed: instead of pointing at the project's `teams/` folders, it adds `code/support/` and `code/` to the Python path. It was checked against the project file: element positions (128), scan-line positions (256), `t_axis` (3574 samples, starting at -0.75 us), depth axis (3118), transmit delays, and all constant values are identical.

run_team5.py is the one-command runner:

1. Copies every printed line to `output/console_log.txt` (class `Tee`) and prints the probe settings.
2. Step 1: runs `code/verify_beamformer.py` as a separate Python process from inside `code/`, prints its output and counts the passed tests.
3. Step 2: loads `input/*.npz` and saves raw-RF plots (full view, zoom with one pixel per sample, one channel).
4. Step 3: computes the delays for the (0, 20) mm target, shows the echo curve before and after the delays and compares `das_beamform` with `apply_focus=True` and `False` on the line $x = 0$.
5. Step 4: beamforms both inputs with `das_beamform` (Hann, $F\# = 1$, 256 lines), saves `beamformed_*.npz` and plots.
6. Step 5: measures the three peak positions and the -6 dB axial and lateral widths at (0, 20) mm, and the mean level inside the cyst. The envelope (Hilbert transform) is used only for display and measuring.
7. Step 6: repeats the beamforming with `rect`, `Hamming` and `Hann` windows, $F\# = 2$ and a fixed full aperture, and compares width, side lobes and a streak level.
8. Saves `console_log.txt` and a screenshot-style picture of the console (`console_screenshot.png`).

3.5 Support files (short description only)

code/support/ holds unchanged copies of other teams' code, used only to make fresh test RF for verify_beamformer.py: Team 3's nitk_usoundsim package (pulse, forward and return propagation with attenuation), Team 4's tissue_interaction.py (reflection) and phantom.py (scatterer phantoms), and tissue_phantoms.py (simulate_rf: joins them and adds element directivity). These are not Team 5's work.

3.6 Full listings

code/receive_beamforming.py

```

1  """
2  receive_beamforming.py
3  =====
4
5  T5 receive delay-and-sum (DAS) beamformer for the NITK-UsoundSim integration.
6
7      das_beamform(rf, element_positions, scanline_positions, config,
8                  apply_focus=True, t_axis=None, apodization=None, f_number=None)
9      -> (beamformed, z_axis)
10
11  Inputs
12  -----
13  rf                : (n_elements, n_samples) one broadcast frame, reused for
14                      every scan line, or (n_elements, n_samples, n_lines), one
15                      frame per line.
16  element_positions : (n_elements,) element x-positions [m], elements at z = 0.
17  scanline_positions : (n_lines,) scan-line x-positions [m].
18  config            : the project config module (depth axis, speed of sound,
19                      default time axis).
20  t_axis            : time axis [s] the RF was sampled on. Defaults to
21                      config.get_t_axis(), which is what tissue_phantoms and
22                      mock_data produce.
23  apodization       : receive window, "hann" | "hamming" | "rect". Defaults to
24                      config.RX_APODIZATION ("hann").
25  f_number          : receive F-number of the dynamic (growing) aperture.
26                      Defaults to config.RX_F_NUMBER (1.0); None = fixed full
27                      aperture with the window across the whole array.
28
29  Outputs
30  -----
31  beamformed : (n_depths, n_lines) beamformed RF -- not an envelope; envelope
32                detection + log compression belong to T6's bmode_formation().
33  z_axis      : (n_depths,) depth [m], config.get_depth_axis() (0 to 60 mm,
34                dz = C / (2 F#)).
35
36  Algorithm (dynamic receive focus, every depth of every scan line)
37  -----
38  For scan line x_l and depth z:
39
40      tau_TX(z)  = min_n sqrt((x_l - x_n)^2 + z^2) / C
41      tau_RX,m(z) = sqrt((x_l - x_m)^2 + z^2) / C
42      s_l(z)     = sum_m w_m(z) * rf_m(tau_TX + tau_RX,m) / sum_m w_m(z)
43
44  tau_TX models the single broadcast transmit (all elements fire together, zero
45  delays): the wave reaches (x_l, z) first from the nearest element. Checked
46  against T3's forward_propagation: the incident envelope peak is within
47  0.05 mm of this for x = 0..10 mm, z = 5..55 mm. w is the receive window across
48  the aperture (Hann by default).
49
50  Dynamic receive aperture (F-number, default 1.0): at depth z only elements
51  with |x_m - x_l| < a(z) = z / (2 F#) receive (at least 2 pitches), and the
52  window is centred on the scan line: w_m(z) = window(|x_m - x_l| / a(z)), e.g.
53  Hann cos^2(pi u / 2). The sum is divided by sum_m w_m(z) so a point target
54  has the same amplitude at every depth. This limits the receive angle to
55  atan(1 / (2 F#)) = 26.6 deg: with the Philips L12-4's coarse pitch
56  (0.30 mm = 1.56 lambda at 8 MHz) steeper angles give grating lobes -- a
57  streak from the shallow point target at -17 dB with the fixed full aperture,
58  below -100 dB with F# = 1 (lateral -6 dB width 0.31 -> 0.47 mm).
59
60  With apply_focus=False, tau_RX,m = z / C for every element -- one
61  fixed delay per depth, no per-element compensation -- so the image is visibly
62  defocused (T5 verification test 2).
```

```

63
64 TIME-AXIS RULE (README Section 6)
65 -----
66 rf_m(tau) is read with np.interp(tau, t_axis, rf[m]) on the REAL t_axis, never
67 tau * FS as an index. t_axis starts at -1.175 us (T3's convention), so
68 tau * FS would put every target ~0.90 mm too deep. Computing tau geometrically
69 does not avoid this by itself -- the lookup is where the bug lives. Delays
70 outside t_axis read as 0.
71
72 Assumptions / limitations
73 -----
74 - The dynamic aperture has a hard edge at a(z) (softened by the window).
75 - Element directivity is applied in the RF simulation (tissue_phantoms), not here.
76 - Straight-ray, constant speed of sound C.
77 ""
78
79 import numpy as np
80
81
82 _WINDOWS = {"hann": np.hanning, "hamming": np.hamming, "rect": np.ones}
83 # Same windows as functions of u = |x_m - x_l| / a(z) in [0, 1) (dynamic aperture)
84 _WINDOW_U = {"hann": lambda u: np.cos(np.pi * u / 2) ** 2,
85              "hamming": lambda u: 0.54 + 0.46 * np.cos(np.pi * u),
86              "rect": lambda u: np.ones_like(u)}
87
88
89 def das_beamform(rf, element_positions, scanline_positions, config, apply_focus=True, t_axis=None,
90                 apodization=None, f_number="config"):
91     """Delay-and-sum receive beamforming; see module docstring."""
92     rf = np.asarray(rf, dtype=float)
93     x_el = np.asarray(element_positions, dtype=float)
94     x_lines = np.atleast_1d(np.asarray(scanline_positions, dtype=float))
95     t_axis = config.get_t_axis() if t_axis is None else np.asarray(t_axis, dtype=float)
96     z_axis = config.get_depth_axis()
97     c = config.SPEED_OF_SOUND
98
99     if rf.ndim == 2:
100         rf = rf[:, :, None] # one broadcast frame, reused for every line
101     if rf.shape[0] != x_el.size or rf.shape[1] != t_axis.size:
102         raise ValueError(
103             f"rf shape {rf.shape[:2]} does not match (n_elements={x_el.size}, len(t_axis)={t_axis.
104             size})"
105         )
106     if rf.shape[2] not in (1, x_lines.size):
107         raise ValueError(f"rf has {rf.shape[2]} frames for {x_lines.size} scan lines")
108
109     apodization = getattr(config, "RX_APODIZATION", "hann") if apodization is None else apodization
110     if apodization not in _WINDOWS:
111         raise ValueError(f"apodization must be one of {sorted(_WINDOWS)}, got {apodization!r}")
112     f_number = getattr(config, "RX_F_NUMBER", None) if f_number == "config" else f_number
113     fixed_weights = _WINDOWS[apodization](x_el.size)
114     if f_number is not None:
115         pitch = np.min(np.diff(np.sort(x_el))) if x_el.size > 1 else 0.0
116         half_aperture = np.maximum(z_axis / (2 * f_number), 2 * pitch)
117     beamformed = np.zeros((z_axis.size, x_lines.size))
118
119     for l, x in enumerate(x_lines):
120         frame = rf[:, :, l if rf.shape[2] > 1 else 0]
121         tau_tx = np.hypot(x - x_el[:, None], z_axis[None, :]).min(axis=0) / c
122         weight_sum = np.zeros(z_axis.size)
123         for m in range(x_el.size):
124             if f_number is None:
125                 w = fixed_weights[m]
126             else:
127                 u = np.abs(x - x_el[m]) / half_aperture
128                 w = np.where(u < 1, _WINDOW_U[apodization](np.minimum(u, 1)), 0.0)
129                 if not w.any():
130                     continue
131                 weight_sum += w
132             tau_rx = np.hypot(x - x_el[m], z_axis) / c if apply_focus else z_axis / c
133             beamformed[:, l] += w * np.interp(
134                 tau_tx + tau_rx, t_axis, frame[m], left=0.0, right=0.0
135             )
136         if f_number is not None:
137             beamformed[:, l] /= np.maximum(weight_sum, 1e-12)
138
139

```

```
140     return beamformed, z_axis
```

code/verify_beamformer.py

```
1  """
2  verify_beamformer.py
3  =====
4  Runs the four T5 verification tests against mock point-target data:
5
6      1. Point-target focusing          (Section 22)
7      2. Correct vs incorrect delay    (Section 23)
8      3. Lateral localization          (Section 24)
9      4. Depth localization            (Section 25)
10
11  Run:  python3 verify_beamformer.py
12  """
13
14  import numpy as np
15
16  import config
17  from mock_data import generate_point_target_rf, generate_two_point_targets_rf
18  from receive_beamforming import das_beamform
19
20
21  def report(label, passed):
22      status = "PASS" if passed else "FAIL"
23      print(f"[{status}] {label}")
24
25
26  def test_1_point_target_focusing():
27      print("\n--- Test 1: Point-target focusing ---")
28      element_positions = config.get_element_positions()
29      x_target, z_target = 0.0, 0.03 # on-axis, 3 cm deep
30
31      rf, _ = generate_point_target_rf(x_target, z_target, element_positions, config)
32      scanline_positions = np.array([x_target]) # beamform straight at the target
33
34      beamformed, z_axis = das_beamform(rf, element_positions, scanline_positions, config)
35
36      peak_idx = np.argmax(np.abs(beamformed))
37      z_peak = z_axis[peak_idx]
38      error_mm = abs(z_peak - z_target) * 1000
39
40      print(f"  expected depth: {z_target*1000:.2f} mm, measured peak: {z_peak*1000:.2f} mm "
41            f"(error {error_mm:.3f} mm)")
42      dz_mm = (config.SPEED_OF_SOUND / (2 * config.SAMPLING_FREQUENCY)) * 1000
43      tolerance_mm = 5 * dz_mm # a few depth-sample widths of slack
44      report(f"peak within {tolerance_mm:.3f} mm tolerance", error_mm < tolerance_mm)
45
46
47  def test_2_correct_vs_incorrect_delay():
48      print("\n--- Test 2: Correct vs incorrect receive delay ---")
49      element_positions = config.get_element_positions()
50      x_target, z_target = 0.0, 0.03
51
52      rf, _ = generate_point_target_rf(x_target, z_target, element_positions, config)
53      scanline_positions = np.array([x_target])
54
55      bf_correct, _ = das_beamform(rf, element_positions, scanline_positions, config, apply_focus=True)
56      bf_wrong, _ = das_beamform(rf, element_positions, scanline_positions, config, apply_focus=False)
57
58      peak_correct = np.max(np.abs(bf_correct))
59      peak_wrong = np.max(np.abs(bf_wrong))
60
61      print(f"  peak amplitude with correct delays: {peak_correct:.3f}")
62      print(f"  peak amplitude with no RX focusing: {peak_wrong:.3f}")
63      report("correct-delay peak is stronger than unfocused peak", peak_correct > peak_wrong)
64
65
66  def test_3_lateral_localization():
67      print("\n--- Test 3: Lateral localization (two targets) ---")
68      element_positions = config.get_element_positions()
69
70      x1, z1 = -0.004, 0.025
```

```

73     x2, z2 = 0.005, 0.035
74
75     rf, _ = generate_two_point_targets_rf(
76         [(x1, z1, 1.0), (x2, z2, 1.0)], element_positions, config
77     )
78
79     scanline_positions = config.get_scanline_positions()
80     # das_beamform expects one RF frame per line; here we reuse the same
81     # plane-wave-style shot for every line (see receive_beamforming.py
82     # docstring re: simplified transmit model).
83     raw_rf_multi = np.repeat(rf[:, :, None], scanline_positions.shape[0], axis=2)
84
85     beamformed, z_axis = das_beamform(raw_rf_multi, element_positions, scanline_positions, config)
86     bmode_like = np.abs(beamformed)
87
88     # Find the two strongest, well-separated local peaks
89     peak_flat = np.argsort(bmode_like, axis=None)[-1:]
90     depth_idx, line_idx = np.unravel_index(peak_flat[0], bmode_like.shape)
91     found_x1, found_z1 = scanline_positions[line_idx], z_axis[depth_idx]
92     print(f"   strongest peak at x={found_x1*1000:.2f} mm, z={found_z1*1000:.2f} mm")
93
94     close_to_1 = abs(found_x1 - x1) < 0.002 and abs(found_z1 - z1) < 0.003
95     close_to_2 = abs(found_x1 - x2) < 0.002 and abs(found_z1 - z2) < 0.003
96     report("strongest peak lands near one of the two targets", close_to_1 or close_to_2)
97
98
99 def test_4_depth_localization():
100     print("\n--- Test 4: Depth localization ---")
101     element_positions = config.get_element_positions()
102     x_target = 0.0
103
104     for z_target in (0.015, 0.04, 0.055):
105         rf, _ = generate_point_target_rf(x_target, z_target, element_positions, config)
106         scanline_positions = np.array([x_target])
107         beamformed, z_axis = das_beamform(rf, element_positions, scanline_positions, config)
108         z_peak = z_axis[np.argmax(np.abs(beamformed))]
109         error_mm = abs(z_peak - z_target) * 1000
110         print(f"   target z={z_target*1000:.1f} mm -> measured peak z={z_peak*1000:.2f} mm "
111               f"(error {error_mm:.3f} mm)")
112         report(f"   depth error small at z={z_target*1000:.1f} mm", error_mm < 0.5)
113
114
115 if __name__ == "__main__":
116     test_1_point_target_focusing()
117     test_2_correct_vs_incorrect_delay()
118     test_3_lateral_localization()
119     test_4_depth_localization()

```

code/mock_data.py

```

1  """
2  mock_data.py
3  =====
4
5  Point-target RF for T5's verify_beamformer.py, generated with the REAL engine,
6  not a simplified model: tissue_phantoms.simulate_rf(), i.e.
7
8      T3 forward_propagation (broadcast, zero TX delays)
9      -> T4 tissue_interaction -> T3 return_propagation
10
11  Both functions return (rf, t_axis): rf has shape (n_elements, len(t_axis)) and
12  t_axis is the real time axis (starts at -1.175 us, T3's convention). Because
13  the RF does not start at t = 0, a beamformer that broke the README Section 6
14  time-axis rule (tau * FS as an index) would land ~0.90 mm too deep and fail
15  test 1 (0.096 mm tolerance).
16  """
17
18 import numpy as np
19
20 from tissue_phantoms import simulate_rf
21
22
23 def _check_geometry(element_positions, config):
24     # simulate_rf() always uses the project array; refuse anything else rather
25     # than silently simulating a different geometry than the caller asked for.
26     if not np.allclose(element_positions, config.get_element_positions()):

```

```

27         raise ValueError("mock_data only supports the project array, config.get_element_positions(
28         ~)")
29
30
31 def generate_point_target_rf(x, z, element_positions, config, amp=1.0):
32     """RF for one point target at (x, z) [m]. Returns (rf, t_axis)."""
33     _check_geometry(element_positions, config)
34     return simulate_rf({"x": x, "z": z, "amp": amp})
35
36
37 def generate_two_point_targets_rf(targets, element_positions, config):
38     """RF for point targets given as [(x, z, amp), ...] [m]. Returns (rf, t_axis)."""
39     _check_geometry(element_positions, config)
40     return simulate_rf({"x": x, "z": z, "amp": a} for x, z, a in targets])

```

code/config.py (added for this package)

```

1  """
2  config.py (local copy for the Team 5 demo package)
3  =====
4
5  This is a local version of the project's simulator/config.py. All numbers and
6  functions are the same as in the project (Philips L12-4 probe: 128 elements,
7  0.30 mm pitch, 8 MHz, fs = 40 MHz, c = 1540 m/s, 256 scan lines over
8  x = +/-10 mm, the same RF time axis and depth axis, Hann receive window,
9  F-number 1.0, element directivity on, ...).
10
11 The ONLY change is where it looks for other code: instead of the project's
12 teams/ folders it uses this package's own folders,
13
14     code/           Team 5's files (receive_beamforming.py, mock_data.py,
15                     verify_beamformer.py) and this config.py
16     code/support/   test-RF generator copied unchanged from other teams
17                     (Team 3's nitk_usoundsim package, Team 4's
18                     tissue_interaction.py / phantom.py, tissue_phantoms.py)
19
20 so the package runs on its own when the folder is copied anywhere.
21
22 Units: metres, seconds, Hz.
23 """
24
25 import sys
26 from pathlib import Path
27
28 import numpy as np
29
30 CODE_DIR = Path(__file__).resolve().parent           # ../code
31 PACKAGE_ROOT = CODE_DIR.parent                       # ../Team_5_RX_Beamforming
32 SUPPORT_DIR = CODE_DIR / "support"
33 INPUT_DIR = PACKAGE_ROOT / "input"
34 OUTPUT_DIR = PACKAGE_ROOT / "output"
35
36 # Make Team 3's package, the support modules and Team 5's modules importable.
37 for _d in (SUPPORT_DIR, CODE_DIR):
38     if str(_d) not in sys.path:
39         sys.path.insert(0, str(_d))
40
41 # -----
42 # Source of truth: T3's flat constants (re-exported unchanged)
43 # -----
44 from nitk_usoundsim.config import ( # noqa: E402
45     ALPHA_0,
46     C,
47     DYNAMIC_RANGE_DB,
48     FS,
49     OUTPUT_SIZE,
50     PHANTOM_POINTS,
51     Z_MAX,
52     Z_MIN,
53 )
54 from nitk_usoundsim.acoustic_propagation import ( # noqa: E402
55     create_linear_array,
56     generate_pulse,
57 )
58
59 # -----

```

```

60 # Probe: Philips L12-4 (FUS4103) reference model
61 # -----
62 # Philips publishes 128 elements and a 4-12 MHz operating range. Pitch and
63 # centre frequency are not published, so T2's documented assumptions are used
64 # (teams/T2_TX_Beamforming/Usound/config/transducer_interface.json:
65 # pitch 0.30 mm, centre 8 MHz). T3's functions default to T3's own 5 MHz, so
66 # every call passes f0=config.F0 explicitly (T3's code is not modified).
67 PROBE_NAME = "Philips L12-4 (FUS4103) reference model"
68 N_ELEMENTS = 128
69 PITCH = 0.30e-3 # [m]; aperture 128 x 0.30 mm = 38.4 mm
70 F0 = 8.0e6 # centre frequency [Hz]
71 F_MIN, F_MAX = 4.0e6, 12.0e6 # published operating range [Hz]
72 LAMBDA = C / F0 # 0.1925 mm; the pitch is 1.56 lambda (not lambda/2)
73 APERTURE = N_ELEMENTS * PITCH
74
75 # -----
76 # Aliases expected by T5's verify_beamformer.py
77 # -----
78 SPEED_OF_SOUND = C
79 SAMPLING_FREQUENCY = FS
80
81
82 def get_element_positions():
83     """Element x-positions [m], shape (N_ELEMENTS,), centred on 0, z = 0 implied (T3 geometry)."""
84     return create_linear_array(N_ELEMENTS, PITCH)
85
86
87 # Imaged lateral FOV: +/-10 mm, well inside the +/-19.2 mm aperture, so every
88 # line is formed under the array (with the 64-element / 9.9 mm array the image
89 # had to be limited to +/-5 mm because of the transmit edge wave).
90 SCANLINE_X_MIN, SCANLINE_X_MAX = -0.010, 0.010
91 N_SCANLINES = 256 # 0.078 mm line spacing (lateral resolution ~0.2 mm)
92
93
94 def get_scanline_positions():
95     """Scan-line x-positions [m], shape (N_SCANLINES,), evenly spanning SCANLINE_X_MIN..SCANLINE_X
96     _MAX."""
97     return np.linspace(SCANLINE_X_MIN, SCANLINE_X_MAX, N_SCANLINES)
98
99
100 # -----
101 # RF time axis and beamforming depth axis
102 # -----
103 # Z_MAX (40 mm) stays the displayed imaging depth. RF is recorded deeper,
104 # because T5's test 4 checks a point target at z = 55 mm.
105 Z_RF_MAX = 0.060
106
107 # T3 convention: t = 0 is the TX pulse centre, and the axis starts one pulse
108 # half-width earlier (t_axis[0] = -0.75 us at 8 MHz). Anything that looks up a
109 # time in the RF must use this axis, never sample_index / FS.
110 _PULSE_T, _ = generate_pulse(f0=F0)
111 T_START = -_PULSE_T[-1]
112
113 # Longest broadcast round trip: farthest element -> deepest, most lateral point -> back.
114 _X_FAR = 0.012 # phantom half-width (PHANTOM_X_MAX below)
115 _D_FAR = np.hypot(_X_FAR + N_ELEMENTS * PITCH / 2, Z_RF_MAX)
116 T_END = 2 * _D_FAR / C + _PULSE_T[-1]
117
118 DZ = C / (2 * FS) # depth step of one RF sample [m]
119
120
121 def get_t_axis():
122     """RF time axis [s], T3 convention (starts at T_START < 0), sampled at FS."""
123     return np.arange(T_START, T_END, 1.0 / FS)
124
125
126 def get_depth_axis():
127     """Beamforming depth axis [m], Z_MIN..Z_RF_MAX in steps of DZ."""
128     return np.arange(Z_MIN, Z_RF_MAX + DZ / 2, DZ)
129
130
131 # -----
132 # Transmit: single broadcast shot (T2 bypassed; T3 used unmodified)
133 # -----
134 TX_FOCUS_Z = None # broadcast, no TX focus. parameter_tests.py tests T2's focused delays separate
135 ly.
136

```

```

137
138 def get_tx_delays():
139     """Per-element TX delays [s] for T3's tau_tx hook: all zero (broadcast)."""
140     return np.zeros(N_ELEMENTS)
141
142
143 # -----
144 # Receive beamforming
145 # -----
146 RX_APODIZATION = "hann" # das_beamform() aperture window: "hann" | "hamming" | "rect"
147 RX_F_NUMBER = 1.0       # dynamic receive aperture z / F#; None = fixed full aperture (see receive
148     ↳ e_beamforming.py)
149
150
151 # -----
152 # Plan Section 5 parameters that T3's engine does not use, recorded here so
153 # they have one home and no module hard-codes a different value.
154 # -----
155 ELEMENT_WIDTH = PITCH          # no kerf assumed; used by the element directivity below
156
157 # Element directivity. T3 models every element as a point that sends and
158 # receives equally in all directions. With the L12-4's coarse pitch
159 # (0.30 mm = 1.56 lambda at 8 MHz) that lets far, steep-angle elements add
160 # late "grating" arrivals: ghosts only -11 to -13 dB below each point target.
161 # A real element of width w is directional; the standard far-field factor of a
162 # rectangular element (as in Field II), sinc(w sin(theta) / lambda), is applied
163 # on transmit and on receive by tissue_phantoms.point_echo() (T3 unmodified).
164 # It lowers those ghosts to -21 to -34 dB.
165 ELEMENT_DIRECTIVITY = True
166 ELEMENT_HEIGHT = None          # elevation not modelled (2-D simulation)
167 BACKGROUND_IMPEDANCE_MRAYL = 1.63 # T4's default (tissue_interaction.py / phantom.py)
168 DENSITY = BACKGROUND_IMPEDANCE_MRAYL * 1e6 / C # ~1058 kg/m^3, from Z = rho * c; informational
169
170
171 # -----
172 # Speckle / cyst phantom settings (passed to T4's PhantomConfig)
173 # -----
174 PHANTOM_SEED = 2026            # T4's default seed
175 PHANTOM_DENSITY_M2 = 1.0e8     # scatterers per m^2 (see README: chosen for fully developed spec
176     ↳ kle at 8 MHz)
177 PHANTOM_X_MIN, PHANTOM_X_MAX = -_X_FAR, _X_FAR # tissue continues 2 mm beyond the imaged lines
178 PHANTOM_Z_MIN, PHANTOM_Z_MAX = Z_MIN, Z_MAX
179
180 # Cyst / lesion at T4's default position and size (30 mm depth, 6 mm radius),
181 # anechoic: scattering scale 0.0, i.e. a fluid-filled cyst (T4's hypoechoic
182 # default is 0.35; None -> T4's default for the echo class).
183 LESION_CENTER_X, LESION_CENTER_Z = 0.0, 0.030
184 LESION_RADIUS_X, LESION_RADIUS_Z = 0.006, 0.006
185 LESION_ECHO_CLASS = "hypoechoic"
186 LESION_SCATTERING_SCALE = 0.0

```

run_team5.py (added for this package)

```

1  """
2  run_team5.py - one-command demo of Team 5: Receive (RX) Beamforming (delay-and-sum)
3
4  Run from this folder:
5      python3 run_team5.py          (macOS / Linux)
6      python  run_team5.py          (Windows)
7
8  What it does
9      Step 1 runs Team 5's verification tests (code/verify_beamformer.py)
10     Step 2 loads the two sample RF inputs from input/ and plots the raw RF
11     Step 3 shows the delay idea for the point target at (0, 20) mm
12     Step 4 beamforms both inputs with das_beamform() and saves the results
13     Step 5 measures target positions and -6 dB resolution at (0, 20) mm
14     Step 6 compares receive apodization windows (rect / hamming / hann) and apertures (F#)
15 Everything is saved to output/ (plots, beamformed data, console log, console screenshot).
16 """
17
18 import subprocess
19 import sys
20 import time
21 from pathlib import Path
22
23 import numpy as np

```

```

24
25 HERE = Path(__file__).resolve().parent
26 CODE = HERE / "code"
27 INPUT = HERE / "input"
28 OUTPUT = HERE / "output"
29 OUTPUT.mkdir(exist_ok=True)
30 sys.path.insert(0, str(CODE)) # Team 5's code + local config.py (config adds code/support)
31
32 import matplotlib # noqa: E402
33
34 matplotlib.use("Agg")
35 import matplotlib.pyplot as plt # noqa: E402
36 from scipy.signal import hilbert # noqa: E402
37
38 import config # noqa: E402 (code/config.py)
39 from receive_beamforming import das_beamform # noqa: E402 (Team 5's beamformer)
40
41
42 # -----
43 # Console log: everything printed goes to the screen AND to output/console_log.txt
44 # -----
45 class Tee:
46     def __init__(self, stream):
47         self.stream, self.lines = stream, []
48
49     def write(self, s):
50         self.stream.write(s)
51         self.lines.append(s)
52
53     def flush(self):
54         self.stream.flush()
55
56     def text(self):
57         return "".join(self.lines)
58
59
60 LOG = Tee(sys.stdout)
61 sys.stdout = LOG
62 T_START = time.time()
63
64
65 def banner(text):
66     print("\n" + "=" * 78)
67     print(text)
68     print("=" * 78)
69
70
71 def save(fig, name):
72     path = OUTPUT / name
73     fig.savefig(path, dpi=130, bbox_inches="tight")
74     plt.close(fig)
75     print(f" saved output/{name}")
76
77
78 def console_png(text, out_png, title="Terminal", width_chars=100, max_lines=160):
79     """Render the captured console text as a screenshot-style PNG (dark background)."""
80     lines = []
81     for line in text.expandtabs(4).rstrip("\n").splitlines():
82         while len(line) > width_chars:
83             lines.append(line[:width_chars])
84             line = " -> " + line[width_chars:]
85         lines.append(line)
86     if len(lines) > max_lines:
87         lines = lines[:max_lines - 1] + [f"... ({len(lines) - max_lines + 1} more lines)"]
88     h = 0.62 + 0.165 * len(lines)
89     fig = plt.figure(figsize=(0.083 * width_chars + 0.4, h))
90     fig.patch.set_facecolor("#1e1e1e")
91     fig.text(0.012, 1 - 0.28 / h, "o o o " + title, color="#9a9a9a", fontsize=9, family="DejaVu
92     ↪ Sans Mono",
93           va="center")
94     for i, ln in enumerate(lines):
95         fig.text(0.012, 1 - (0.55 + 0.165 * i) / h, ln, color="#e6e6e6", fontsize=8.6, family="Dej
96     ↪ aVu Sans Mono",
97           va="top")
98     fig.savefig(out_png, dpi=160, facecolor=fig.get_facecolor())
99     plt.close(fig)
100

```

```

101
102 def envelope_db(bf, ref=None):
103     """Envelope (Hilbert, along depth) in dB. FOR DISPLAY / MEASUREMENT ONLY (next team's job)."""
104     env = np.abs(hilbert(bf, axis=0))
105     ref = env.max() if ref is None else ref
106     return env, 20 * np.log10(np.maximum(env / ref, 1e-12))
107
108
109 def width_at(profile, axis, i_peak, level=0.5):
110     """Full width where profile >= level * peak, with linear interpolation at both crossings."""
111     p = profile / profile[i_peak]
112     i = i_peak
113     while i > 0 and p[i] >= level:
114         i -= 1
115     left = axis[i] + (level - p[i]) * (axis[i + 1] - axis[i]) / (p[i + 1] - p[i])
116     j = i_peak
117     while j < len(p) - 1 and p[j] >= level:
118         j += 1
119     right = axis[j - 1] + (p[j - 1] - level) * (axis[j] - axis[j - 1]) / (p[j - 1] - p[j])
120     return right - left
121
122
123 # -----
124 banner("NITK-UsoundSim | Team 5: Receive (RX) Beamforming, delay-and-sum")
125 print(f"Probe : {config.PROBE_NAME}")
126 print(f" {config.N_ELEMENTS} elements, pitch {config.PITCH*1e3:.2f} mm, f0 {config.F0/1e6}
127     ↪ :.0f} MHz, "
128     f"fs {config.FS/1e6:.0f} MHz, c {config.C:.0f} m/s")
129 print(f"Lines : {config.N_SCANLINES} scan lines, x = {config.SCANLINE_X_MIN*1e3:+.0f} .. "
130     f"{config.SCANLINE_X_MAX*1e3:+.0f} mm")
131 print(f"Receive : apodization '{config.RX_APODIZATION}', F-number {config.RX_F_NUMBER}")
132 print(f"Python : {sys.version.split()[0]} numpy {np.__version__}")
133 import receive_beamforming # noqa: E402
134
135 print(f"Beamformer loaded from: {Path(receive_beamforming.__file__).relative_to(HERE)}")
136
137 # -----
138 banner("STEP 1 Team 5 verification tests (code/verify_beamformer.py)")
139 print("Test RF is generated fresh by mock_data.py -> support/tissue_phantoms.simulate_rf ...\n")
140 t0 = time.time()
141 res = subprocess.run([sys.executable, "verify_beamformer.py"], cwd=CODE, capture_output=True, text
142     ↪ =True)
143 out = res.stdout.strip("\n")
144 print(out)
145 if res.returncode != 0:
146     print(res.stderr)
147     print(" !! verify_beamformer.py exited with an error (see above)")
148 n_pass, n_fail = out.count("[PASS]"), out.count("[FAIL]")
149 test_blocks = out.split("--- Test ")[1:]
150 n_tests_passed = sum(1 for b in test_blocks if "[PASS]" in b and "[FAIL]" not in b)
151 print(f"\nSummary: {n_tests_passed}/{len(test_blocks)} tests passed "
152     f"({n_pass} PASS checks, {n_fail} FAIL checks) [{time.time()-t0:.1f} s]")
153
154 # -----
155 banner("STEP 2 Load the sample inputs (input/*.npz)")
156 inputs = {}
157 for name, file, desc in [
158     ("point_targets", "point_targets_rf.npz", "3 point targets at (-5,10), (0,20), (+5,30) mm"),
159     ("cyst", "cyst_rf.npz", "speckle tissue with an anechoic cyst, centre (0,30) mm, radius 6 mm")
160     ↪ ,
161 ]:
162     d = np.load(INPUT / file)
163     rf, t_axis = d["rf"].astype(float), d["t_axis"]
164     inputs[name] = (rf, t_axis)
165     print(f"{file:22s}: {desc}")
166     print(f"{'':22s} rf {rf.shape} (elements x time samples), |rf| max {np.abs(rf).max():.3g}")
167     print(f"{'':22s} t_axis {t_axis[0]*1e6:+.3f} .. {t_axis[-1]*1e6:.3f} us, step {np.diff(t_axis
168     ↪ ).mean()*1e9:.1f} ns "
169     f" (= 1/fs)")
170
171 element_x = config.get_element_positions()
172 x_lines = config.get_scanline_positions()
173
174
175 def plot_raw(name, title, zoom_t, trace_el=64):
176     rf, t_axis = inputs[name]
177     t_us = t_axis * 1e6

```

```

178     lim = np.abs(rf).max()
179     fig, ax = plt.subplots(1, 3, figsize=(15, 5.2), gridspec_kw={"width_ratios": [1.1, 1, 1.3]})
180     ax[0].imshow(rf.T, aspect="auto", cmap="gray", vmin=-lim / 3, vmax=lim / 3,
181                 extent=[0.5, rf.shape[0] + 0.5, t_us[-1], t_us[0]], interpolation="none")
182     ax[0].set(xlabel="element number (1..128)", ylabel="time t [us]", title="Full raw RF (one broad
183     ↳ dcast shot)")
184     ax[0].add_patch(plt.Rectangle((trace_el - 12, zoom_t[0]), 24, zoom_t[1] - zoom_t[0], fill=False
185     ↳ e, ec="r", lw=1.5))
186     it = (t_us >= zoom_t[0]) & (t_us <= zoom_t[1])
187     sub = rf[trace_el - 13:trace_el + 11, it]
188     ax[1].imshow(sub.T, aspect="auto", cmap="seismic", vmin=-np.abs(sub).max(), vmax=np.abs(sub).m
189     ↳ ax(),
190                 extent=[trace_el - 12.5, trace_el + 11.5, t_us[it][-1], t_us[it][0]], interpolati
191     ↳ on="none")
192     ax[1].set(xlabel="element number", ylabel="time t [us]", title="Zoom (red box): each pixel = o
193     ↳ ne sample")
194     ax[2].plot(t_us[it], rf[trace_el - 1, it], "k-", lw=0.9, marker=".", ms=3)
195     ax[2].set(xlabel="time t [us]", ylabel="amplitude", title=f"One channel: element {trace_el} (d
196     ↳ ots = samples)")
197     ax[2].grid(alpha=0.3)
198     fig.suptitle(title, fontsize=12)
199     fig.tight_layout()
200     return fig
201
202
203 print()
204 save(plot_raw("point_targets", "INPUT: raw RF of the point-target phantom (before beamforming)", (
205     ↳ 25.4, 27.4)),
206     "01_input_point_targets_raw_rf.png")
207 save(plot_raw("cyst", "INPUT: raw RF of the cyst phantom (speckle; before beamforming)", (38.0, 40
208     ↳ .0)),
209     "02_input_cyst_raw_rf.png")
210
211 # -----
212 banner("STEP 3 The delay idea for the target at (0, 20) mm")
213 rf, t_axis = inputs["point_targets"]
214 c = config.C
215 xl, zt = 0.0, 0.020
216 tau_tx = np.hypot(xl - element_x, zt).min() / c
217 tau_rx = np.hypot(xl - element_x, zt) / c
218 arrival = tau_tx + tau_rx
219 print(f"tau_TX at (0,20) mm = {tau_tx*1e6:.3f} us (nearest element fires first, broadcast)")
220 print(f"tau_TX + tau_RX: centre element {arrival[63]*1e6:.3f} us, edge element 1 {arrival[0]*1e6:.
221     ↳ 3f} us")
222 print(f" -> the echo reaches the edge element {(arrival[0]-arrival[63])*1e6:.2f} us later than th
223     ↳ e centre")
224 z_win = np.arange(0.0185, 0.0215, config.DZ)
225 aligned = np.zeros((element_x.size, z_win.size))
226 for m in range(element_x.size):
227     tt = np.hypot(xl - element_x, z_win[:, None]).min(axis=1) / c + np.hypot(xl - element_x[m], z_
228     ↳ win) / c
229     aligned[m] = np.interp(tt, t_axis, rf[m], left=0.0, right=0.0)
230 line_f, z_axis = das_beamform(rf, element_x, [xl], config, t_axis=t_axis)
231 line_u, _ = das_beamform(rf, element_x, [xl], config, t_axis=t_axis, apply_focus=False)
232 print(f"peak |RF| on line x=0 near 20 mm: with RX delays {np.abs(line_f[:, 0]).max():.3f}, "
233     f"without per-element delays {np.abs(line_u[:, 0]).max():.3f}")
234
235 t_us = t_axis * 1e6
236 it = (t_us > 24.5) & (t_us < 32.5)
237 fig, ax = plt.subplots(1, 3, figsize=(15, 5.2), gridspec_kw={"width_ratios": [1, 1, 1.1]})
238 lim = np.abs(rf[:, it]).max() / 2
239 ax[0].imshow(rf[:, it].T, aspect="auto", cmap="seismic", vmin=-lim, vmax=lim, interpolation="none"
240     ↳ ,
241                 extent=[0.5, 128.5, t_us[it][-1], t_us[it][0]])
242 ax[0].plot(np.arange(1, 129), arrival * 1e6, "k--", lw=1, label="tau_TX + tau_RX,m")
243 ax[0].set(xlabel="element number", ylabel="time t [us]", title="BEFORE delays: echo is a curve")
244 ax[0].legend(loc="lower center", fontsize=8)
245 lim2 = np.abs(aligned).max() / 2
246 ax[1].imshow(aligned.T, aspect="auto", cmap="seismic", vmin=-lim2, vmax=lim2, interpolation="none"
247     ↳ ,
248                 extent=[0.5, 128.5, z_win[-1] * 1e3, z_win[0] * 1e3])
249 ax[1].axhline(20, color="k", ls="--", lw=0.8)
250 ax[1].set(xlabel="element number", ylabel="depth z [mm]", title="AFTER delays: echo is a straight
251     ↳ line")
252 zz = (z_axis > 0.0185) & (z_axis < 0.0215)
253 ax[2].plot(z_axis[zz] * 1e3, line_f[zz, 0], "b-", lw=1.2, label="with RX delays (focused)")
254 ax[2].plot(z_axis[zz] * 1e3, line_u[zz, 0], "r-", lw=1.0, label="no per-element delay (apply_focus

```

```

255     =False)")
256 ax[2].set(xlabel="depth z [mm]", ylabel="beamformed RF", title="SUM over elements, scan line x = 0
257     ")
258 ax[2].legend(fontsize=8)
259 ax[2].grid(alpha=0.3)
260 fig.suptitle("Delay-and-sum for the point target at (0, 20) mm (elements outside the F#=1 aperture
261     get weight 0 "
262         "in the sum)", fontsize=11)
263 fig.tight_layout()
264 save(fig, "03_delay_alignment_0_20mm.png")
265
266 # -----
267 banner("STEP 4 Beamform both inputs with das_beamform() (hann, F# = 1, 256 lines)")
268 results = {}
269 for name in ("point_targets", "cyst"):
270     rf, t_axis = inputs[name]
271     t0 = time.time()
272     bf, z_axis = das_beamform(rf, element_x, x_lines, config, t_axis=t_axis)
273     results[name] = bf
274     print(f"{name:14s}: output {bf.shape} (depths x lines), z {z_axis[0]*1e3:.0f}...{z_axis[-1]*1e3
275     :.0f} mm "
276         f"step {config.DZ*1e3:.4f} mm, {time.time()-t0:.1f} s")
277     np.savez_compressed(OUTPUT / f"beamformed_{name}.npz", beamformed=bf.astype(np.float32), z_axi
278     s=z_axis,
279         x_lines=x_lines)
280     print(f"'':14s} saved output/beamformed_{name}.npz (Team 5's output for the next stage)")
281
282 ext = [x_lines[0] * 1e3, x_lines[-1] * 1e3, 40, 0]
283 zshow = z_axis <= 0.040 + 1e-9
284 for k, (name, label) in enumerate([("point_targets", "point targets"), ("cyst", "cyst phantom")]):
285     bf = results[name]
286     # colour scale: points clipped at 1/3 of the peak so weaker echoes show; speckle at its 99.5th
287     percentile
288     lim = np.abs(bf[zshow]).max() / 3 if name == "point_targets" else np.percentile(np.abs(bf[zsho
289     w]), 99.5)
290     fig, ax = plt.subplots(1, 2, figsize=(12, 7), gridspec_kw={"width_ratios": [1, 1]})
291     # overview: red/blue for the sparse point targets; grey for dense speckle (red/blue would blur
292     to purple)
293     ax[0].imshow(bf[zshow], cmap="seismic" if name == "point_targets" else "gray", vmin=-lim, vmax
294     =lim, extent=ext,
295         aspect="equal", interpolation="none")
296     ax[0].set(xlabel="lateral x [mm]", ylabel="depth z [mm]", title=f"Beamformed RF, {label} (0-40
297     mm shown)")
298     if name == "point_targets":
299         mz = np.abs(z_axis - 0.020) < 0.0012 # zoom around the (0, 20) mm target
300     else:
301         mz = np.abs(z_axis - 0.0235) < 0.0012 # zoom into speckle above the cyst
302         mx = np.abs(x_lines) < 0.0012
303         sub = bf[np.ix_(mz, mx)]
304         ax[1].imshow(sub, cmap="seismic", vmin=-np.abs(sub).max(), vmax=np.abs(sub).max(), interpolati
305         on="none",
306             extent=[x_lines[mx][0] * 1e3, x_lines[mx][-1] * 1e3, z_axis[mz][-1] * 1e3, z_axis
307             [mz][0] * 1e3],
308             aspect="auto")
309         ax[1].set(xlabel="lateral x [mm]", ylabel="depth z [mm]",
310             title="Zoom: RF still oscillates along depth (not an envelope)")
311         ax[0].add_patch(plt.Rectangle((x_lines[mx][0] * 1e3, z_axis[mz][0] * 1e3), (x_lines[mx][-1] -
312             x_lines[mx][0]) * 1e3,
313             (z_axis[mz][-1] - z_axis[mz][0]) * 1e3, fill=False, ec="k", lw=1
314             .2))
315         fig.suptitle(f"OUTPUT of Team 5: beamformed RF, {label} (signed RF: red/white = +, blue/black
316             = -)",
317             fontsize=12)
318         fig.tight_layout()
319         save(fig, f"{4 + 2*k:02d}_beamformed_rf_{name}.png")
320
321     env, db = envelope_db(bf)
322     fig, ax = plt.subplots(figsize=(6.2, 8))
323     im = ax.imshow(db[zshow], cmap="gray", vmin=-50, vmax=0, extent=ext, aspect="equal", interpola
324     tion="none")
325     ax.set(xlabel="lateral x [mm]", ylabel="depth z [mm]",
326         title=f"{label}: display image (50 dB)\nFOR DISPLAY ONLY: envelope + log are the next t
327     eam's job")
328     if name == "point_targets":
329         for (tx_, tz_) in [(-5, 10), (0, 20), (5, 30)]:
330             ax.plot(tx_, tz_, "o", mfc="none", mec="c", ms=14, mew=1)
331     else:

```

```

332     th = np.linspace(0, 2 * np.pi, 200)
333     ax.plot(6 * np.cos(th), 30 + 6 * np.sin(th), "c--", lw=0.8)
334     plt.colorbar(im, ax=ax, label="dB (0 = brightest)", shrink=0.7)
335     fig.tight_layout()
336     save(fig, f"{5 + 2*k:02d}_display_{name}.png")
337
338 # -----
339 banner("STEP 5 Measurements on the beamformed point-target image")
340 bf = results["point_targets"]
341 env, _ = envelope_db(bf)
342 print("(peaks are taken from the envelope of the beamformed RF;")
343 print(f" grid step dz = {config.DZ*1e3:.4f} mm, dx = {np.diff(x_lines).mean()*1e3:.4f} mm)")
344 print(f"{'true (x, z) [mm]':>18s} | {'measured (x, z) [mm]':>21s} | {'error x, z [mm]':>16s} | pea
345     ↳ k level")
346 peak_ref = env.max()
347 for tx_, tz_ in [(-5, 10), (0, 20), (5, 30)]:
348     m = (np.abs(z_axis[:, None] * 1e3 - tz_) <= 1.5) & (np.abs(x_lines[None, :] * 1e3 - tx_) <= 1.
349     ↳ 5)
350     e = np.where(m, env, 0)
351     iz, il = np.unravel_index(np.argmax(e), e.shape)
352     mx_, mz_ = x_lines[il] * 1e3, z_axis[iz] * 1e3
353     print(f"{'({tx_:+.0f}, {tz_:.0f})':>18s} | {'({mx_:+.3f}, {mz_:.3f})':>21s} | "
354     f"{'({mx_-tx_:+.3f}, {mz_-tz_:+.3f})':>16s} | {20*np.log10(env[iz, il]/peak_ref):6.1f} dB
355     ↳ ")
356
357 m = (np.abs(z_axis[:, None] - 0.020) <= 0.0015) & (np.abs(x_lines[None, :]) <= 0.0015)
358 iz, il = np.unravel_index(np.argmax(np.where(m, env, 0)), env.shape)
359 ax_w = width_at(env[:, il], z_axis, iz) * 1e3
360 lat_w = width_at(env[iz, :], x_lines, il) * 1e3
361 print(f"\n-6 dB resolution at the (0, 20) mm target (peak at x={x_lines[il]*1e3:+.3f}, z={z_axis[i
362     ↳ z]*1e3:.3f} mm):")
363 print(f" axial width (along depth) : {ax_w:.3f} mm")
364 print(f" lateral width (across lines): {lat_w:.3f} mm")
365 print(f" (wavelength = {config.LAMBDA*1e3:.4f} mm; lateral line spacing {np.diff(x_lines).mean()*
366     ↳ 1e3:.4f} mm)")
367
368 fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
369 za = (z_axis > z_axis[iz] - 0.001) & (z_axis < z_axis[iz] + 0.001)
370 ax[0].plot(z_axis[za] * 1e3, bf[za, il] / env[iz, il], color="0.6", lw=0.9, label="beamformed RF")
371 ax[0].plot(z_axis[za] * 1e3, env[za, il] / env[iz, il], "b-", lw=1.5, label="envelope")
372 ax[0].axhline(0.5, color="r", ls="--", lw=0.8, label="-6 dB (half amplitude)")
373 ax[0].set(xlabel="depth z [mm]", ylabel="normalised amplitude", title=f"Axial profile: -6 dB width
374     ↳ {ax_w:.3f} mm")
375 ax[0].legend(fontsize=8)
376 ax[0].grid(alpha=0.3)
377 xa = np.abs(x_lines) < 0.002
378 ax[1].plot(x_lines[xa] * 1e3, env[iz, xa] / env[iz, il], "b-", lw=1.5, label="envelope at peak de
379     ↳ pth")
380 ax[1].axhline(0.5, color="r", ls="--", lw=0.8, label="-6 dB (half amplitude)")
381 ax[1].set(xlabel="lateral x [mm]", ylabel="normalised amplitude",
382     title=f"Lateral profile: -6 dB width {lat_w:.3f} mm")
383 ax[1].legend(fontsize=8)
384 ax[1].grid(alpha=0.3)
385 fig.suptitle("Resolution at the point target (0, 20) mm", fontsize=12)
386 fig.tight_layout()
387 save(fig, "08_resolution_profiles_0_20mm.png")
388
389 bfc = results["cyst"]
390 envc = np.abs(hilbert(bfc, axis=0))
391 R = np.hypot(x_lines[None, :] - 0.0, z_axis[:, None] - 0.030)
392 inside = R < 0.004
393 ring = (R > 0.0075) & (R < 0.0105) & (z_axis[:, None] < 0.040)
394 ratio = 20 * np.log10(envc[inside].mean() / envc[ring].mean())
395 print(f"\nCyst image: mean envelope inside r < 4 mm of (0,30) mm = {ratio:.1f} dB")
396 print(" relative to the surrounding tissue ring (r = 7.5-10.5 mm)")
397
398 # -----
399 banner("STEP 6 Apodization / aperture comparison at the (0, 20) mm target")
400 rf, t_axis = inputs["point_targets"]
401 cases = [("rect", "config"), ("hamming", "config"), ("hann", "config"), ("hann", 2.0), ("hann", No
402     ↳ ne)]
403 zwin = np.abs(z_axis - 0.020) <= 0.001
404 xwin = np.abs(x_lines) <= 0.005
405 print("lateral width: -6 dB width across lines at the peak depth")
406 print("highest side lobe: max over depth 19-21 mm of the envelope, outside the main lobe, |x| <= 5
407     ↳ mm")
408 print("streak: brightest point right of the (-5,10) mm target (x > 1 mm, z 9.5-16 mm),")

```

```

409 print("          in dB relative to the image maximum")
410 print(f"{'window':>9s} {'aperture':>16s} | {'lateral -6 dB':>13s} | {'highest side lobe':>17s} | {
411     ↳ 'streak':>8s}")
412 streak_zone = (z_axis[:, None] > 0.0095) & (z_axis[:, None] < 0.016) & (x_lines[None, :] > 0.001)
413 profiles = []
414 for apod, fnum in cases:
415     bf_a, _ = das_beamform(rf, element_x, x_lines, config, t_axis=t_axis, apodization=apod, f_num=
416     ↳ fnum)
417     env_a = np.abs(hilbert(bf_a, axis=0))
418     mm = zwin[:, None] & (np.abs(x_lines[None, :]) <= 0.0015)
419     iz_a, il_a = np.unravel_index(np.argmax(np.where(mm, env_a, 0)), env_a.shape)
420     w = width_at(env_a[iz_a, :], x_lines, il_a) * 1e3
421     proj = env_a[zwin].max(axis=0)
422     proj_db = 20 * np.log10(proj / env_a[iz_a, il_a])
423     # main lobe: walk out from the peak while the projection keeps falling
424     a = il_a
425     while a > 0 and proj[a - 1] < proj[a]:
426         a -= 1
427     b = il_a
428     while b < len(proj) - 1 and proj[b + 1] < proj[b]:
429         b += 1
430     outside = xwin.copy()
431     outside[a:b + 1] = False
432     sl = proj_db[outside].max()
433     streak = 20 * np.log10(np.where(streak_zone, env_a, 0).max() / env_a.max())
434     label = {"config": "F# = 1 (dynamic)", 2.0: "F# = 2 (dynamic)", None: "full, fixed"}[fnum]
435     print(f"{apod:>9s} {label:>16s} | {w:10.3f} mm | {sl:14.1f} dB | {streak:5.1f} dB")
436     profiles.append((f"{apod}, {label}", x_lines, proj_db))
437
438 fig, ax = plt.subplots(1, 2, figsize=(13, 4.4), gridspec_kw={"width_ratios": [1, 1.6]})
439 u = np.linspace(-1, 1, 201)
440 for nm, f in [("rect", lambda u: np.ones_like(u)), ("hamming", lambda u: 0.54 + 0.46 * np.cos(np.p
441     ↳ i * u)),
442     ↳ ("hann", lambda u: np.cos(np.pi * u / 2) ** 2)]:
443     ax[0].plot(u, f(np.abs(u)), lw=1.5, label=nm)
444 ax[0].set(xlabel="(x_m - x_l) / a(z) (position inside the receive aperture)", ylabel="weight w",
445     title="Receive apodization windows")
446 ax[0].legend()
447 ax[0].grid(alpha=0.3)
448 for (lab, xx, pdb), st in zip(profiles, ["-", "-.", ".", "--", ":"]):
449     ax[1].plot(xx * 1e3, pdb, st, lw=1.2, label=lab)
450 ax[1].set(xlim=(-5, 5), ylim=(-80, 3), xlabel="lateral x [mm]", ylabel="dB",
451     title="Lateral profile at (0, 20) mm (max over depth 19-21 mm)")
452 ax[1].legend(fontsize=8)
453 ax[1].grid(alpha=0.3)
454 fig.tight_layout()
455 save(fig, "09_apodization_comparison.png")
456
457 # -----
458 banner("DONE")
459 print(f"Verification tests: {n_tests_passed}/{len(test_blocks)} passed")
460 print(f"Total run time: {time.time()-T_START:.1f} s")
461 print("All outputs are in: output/ (9 plots + console_screenshot.png, 2 beamformed .npz, console_
462     ↳ log.txt)")
463 text = LOG.text()
464 (OUTPUT / "console_log.txt").write_text(text)
465 console_png(text, OUTPUT / "console_screenshot.png", title="python3 run_team5.py")
466 sys.stdout = LOG.stream
467 print(" saved output/console_log.txt and output/console_screenshot.png")

```

4 Input

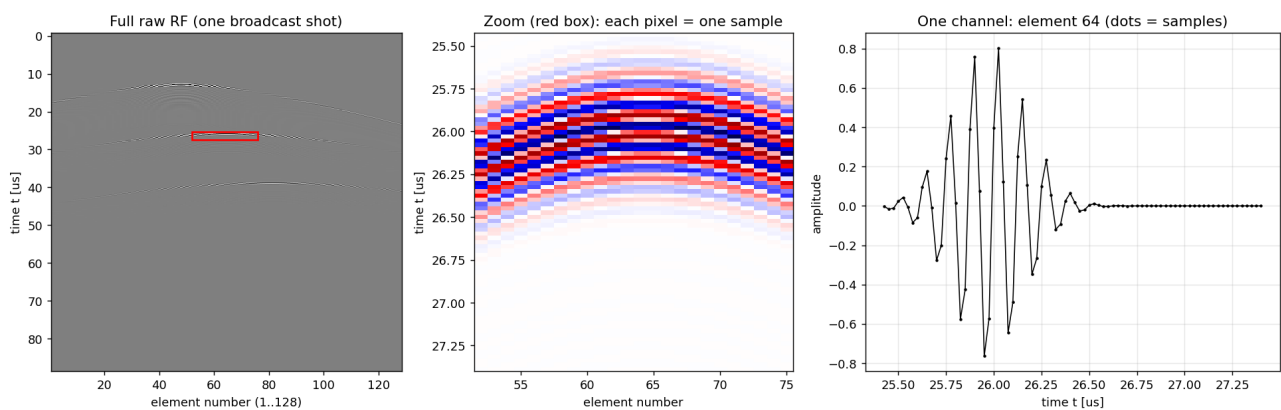
The sample raw RF given to the beamformer

The folder contains two sample inputs. Both are real simulated RF saved by the project's full simulator run (propagation + tissue stages, one broadcast transmit, all 128 elements). Only two arrays are kept in each file:

File	Arrays	What is inside
input/point_targets_rf.npz	rf (128, 3574) float32, t_axis (3574,) float64	Three point targets at (x, z) = (-5, 10), (0, 20) and (+5, 30) mm, amplitude 1.
input/cyst_rf.npz	rf (128, 3574) float32, t_axis (3574,) float64	Speckle tissue (many random scatterers, x = -12..+12 mm, z = 0..40 mm) with an anechoic (no echo) circular cyst, centre (0, 30) mm, radius 6 mm.

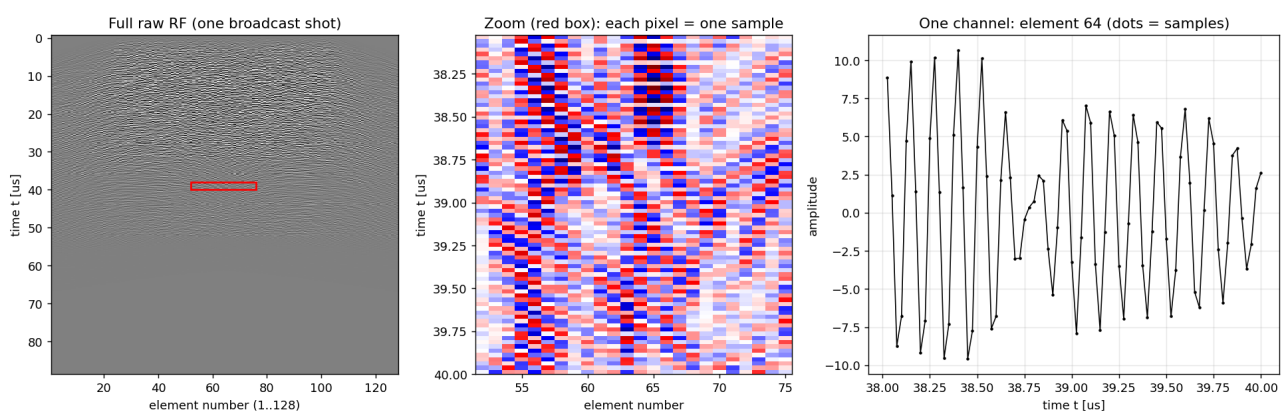
$rf[m, k]$ is the signal of element m at time $t_axis[k]$. t_axis runs from -0.750 us to 88.575 us in steps of 25 ns (1/fs). The RF was stored as float32 to keep the files small; beamforming this float32 RF gave the same image as the project's float64 result within a relative difference of about $1e-8$.

INPUT: raw RF of the point-target phantom (before beamforming)



Raw RF of the point targets. Left: all elements against time; each point gives a curved echo (10 mm target near 13 us, 20 mm near 26 us, 30 mm near 39 us). Middle: zoom into the red box, one pixel per sample (no smoothing), showing the oscillating 8 MHz echo. Right: one channel (element 64); the dots are the real samples.

INPUT: raw RF of the cyst phantom (speckle; before beamforming)



Raw RF of the cyst phantom. The echoes of thousands of scatterers overlap, so no single curve is visible. The cyst cannot be seen in raw channel data; it appears only after beamforming.

5 Output

What run_team5.py produced (all numbers from the real run)

5.1 Verification tests

Team 5 tests (from output/console_log.txt)

```
Test RF is generated fresh by mock_data.py -> support/tissue_phantoms.simulate_rf ...
```

```
--- Test 1: Point-target focusing ---
  expected depth: 30.00 mm, measured peak: 29.99 mm (error 0.008 mm)
[PASS] peak within 0.096 mm tolerance
```

```
--- Test 2: Correct vs incorrect receive delay ---
  peak amplitude with correct delays: 0.327
  peak amplitude with no RX focusing: 0.066
[PASS] correct-delay peak is stronger than unfocused peak
```

```
--- Test 3: Lateral localization (two targets) ---
  strongest peak at x=-3.96 mm, z=24.99 mm
[PASS] strongest peak lands near one of the two targets
```

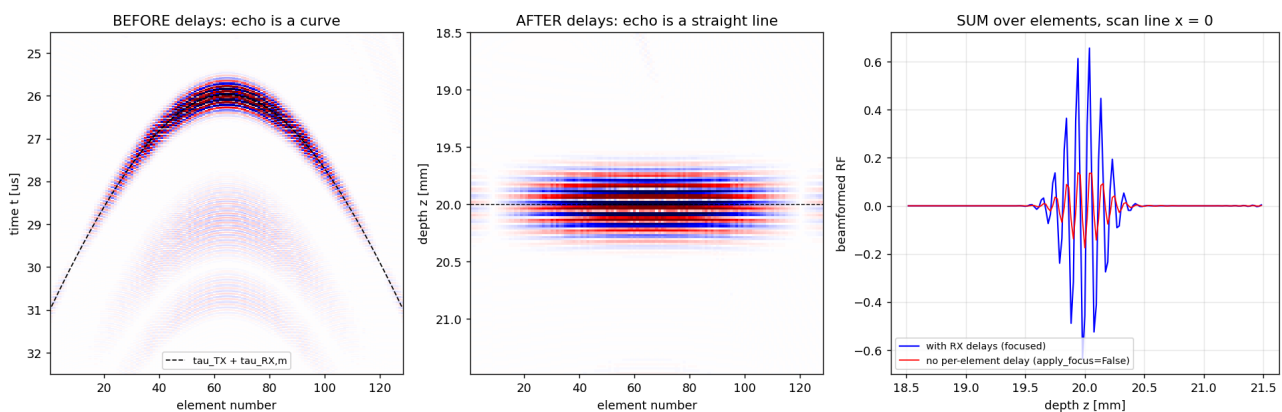
```
--- Test 4: Depth localization ---
  target z=15.0 mm -> measured peak z=15.03 mm (error 0.034 mm)
[PASS] depth error small at z=15.0 mm
  target z=40.0 mm -> measured peak z=40.04 mm (error 0.040 mm)
[PASS] depth error small at z=40.0 mm
  target z=55.0 mm -> measured peak z=55.04 mm (error 0.036 mm)
[PASS] depth error small at z=55.0 mm
```

```
Summary: 4/4 tests passed (6 PASS checks, 0 FAIL checks) [28.5 s]
```

All 4 tests pass. Depth errors are 0.008 mm (30 mm), 0.034 mm (15 mm), 0.040 mm (40 mm) and 0.036 mm (55 mm), i.e. about two depth samples or less. Correct receive delays give a peak about 5 times stronger than no per-element delay (0.327 vs 0.066).

5.2 The delay idea

Delay-and-sum for the point target at (0, 20) mm (elements outside the F#=1 aperture get weight 0 in the sum)

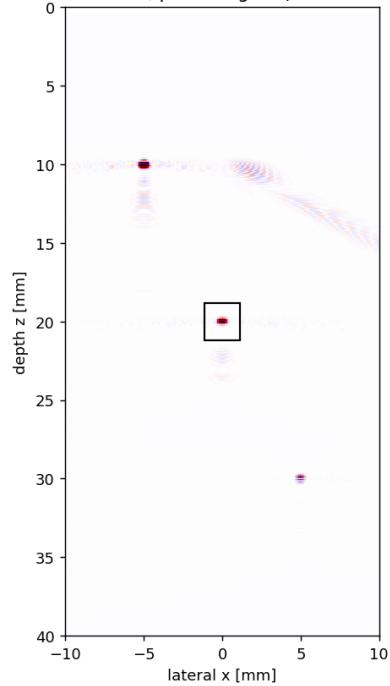


Left: raw RF around the (0, 20) mm echo with the computed arrival time $\tau_{TX} + \tau_{RX,m}$ (dashed); the curve lies on the echo. Middle: the same channels read at the delayed times for depths 18.5-21.5 mm; the echo is now a straight line at 20 mm. Right: the beamformed line $x = 0$ with the correct delays (peak 0.656) and with one delay for all elements (peak 0.175).

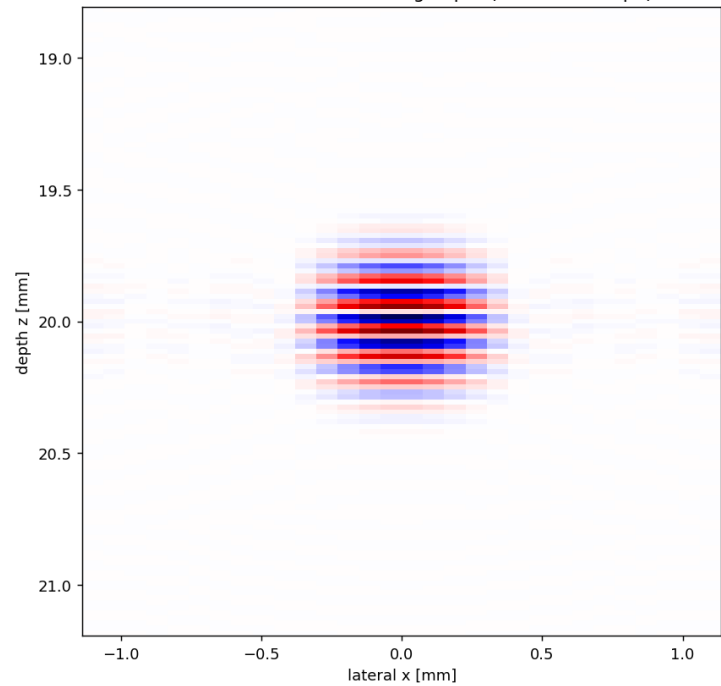
5.3 Beamformed RF and display images

OUTPUT of Team 5: beamformed RF, point targets (signed RF: red/white = +, blue/black = -)

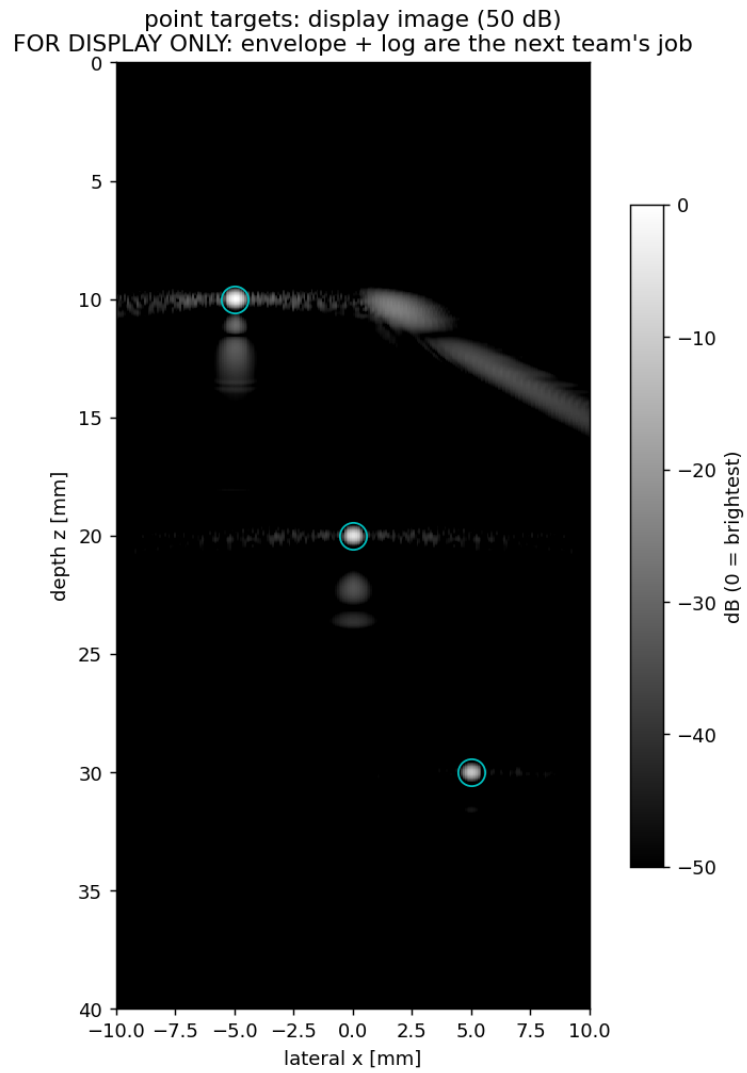
Beamformed RF, point targets (0-40 mm shown)



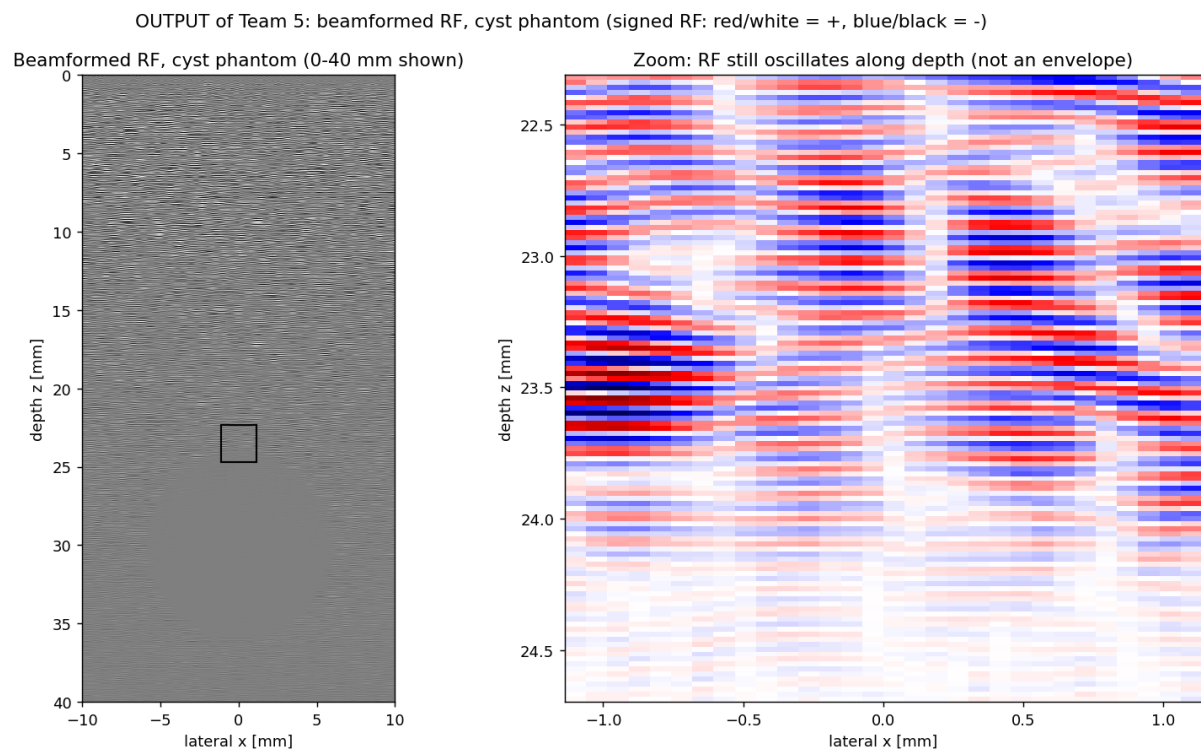
Zoom: RF still oscillates along depth (not an envelope)



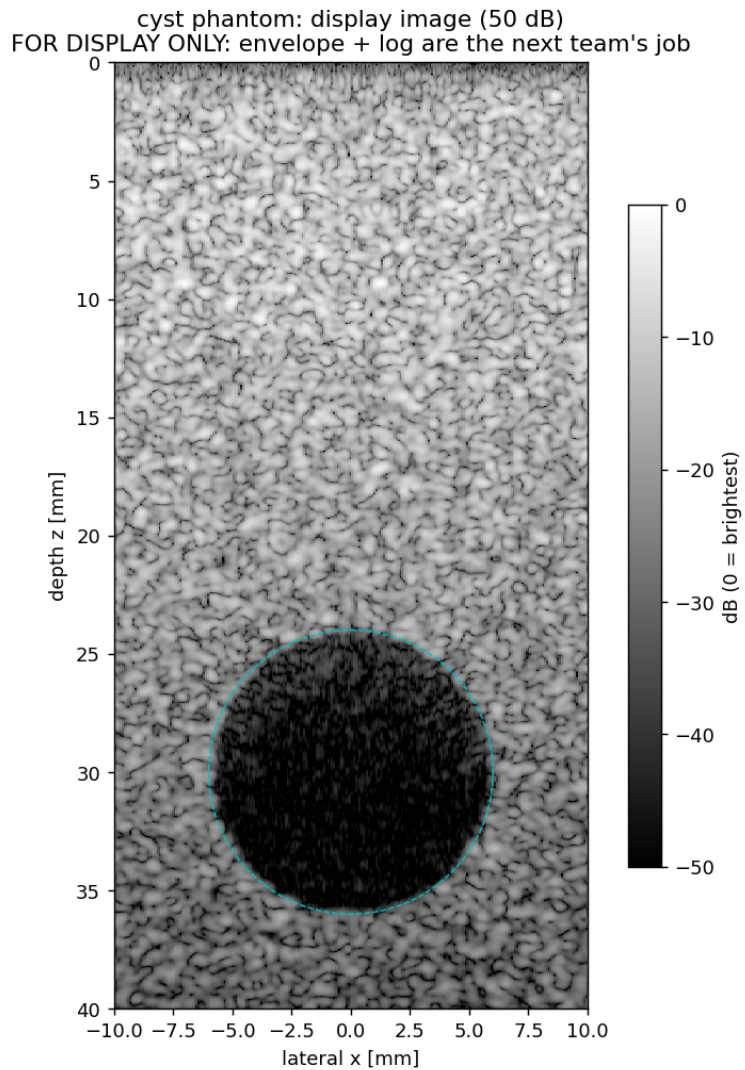
Team 5's output for the point targets (3118 x 256, shown 0-40 mm). Each target is now a small spot at the right place. The zoom shows the RF still oscillates along depth: it is not yet an envelope.



Display image of the same result (envelope + $20 \log_{10}$, 50 dB range). FOR DISPLAY ONLY: envelope and log compression are the next team's job. Circles mark the true positions. A faint tilted streak right of the 10 mm target and weak marks below the targets are visible at about -25 dB or lower (see 5.5).



Beamformed RF of the cyst phantom. In the signed RF (grey scale) the cyst is hard to see, only a slightly quieter area at 24-36 mm; the zoom shows the oscillating speckle RF just above the cyst.

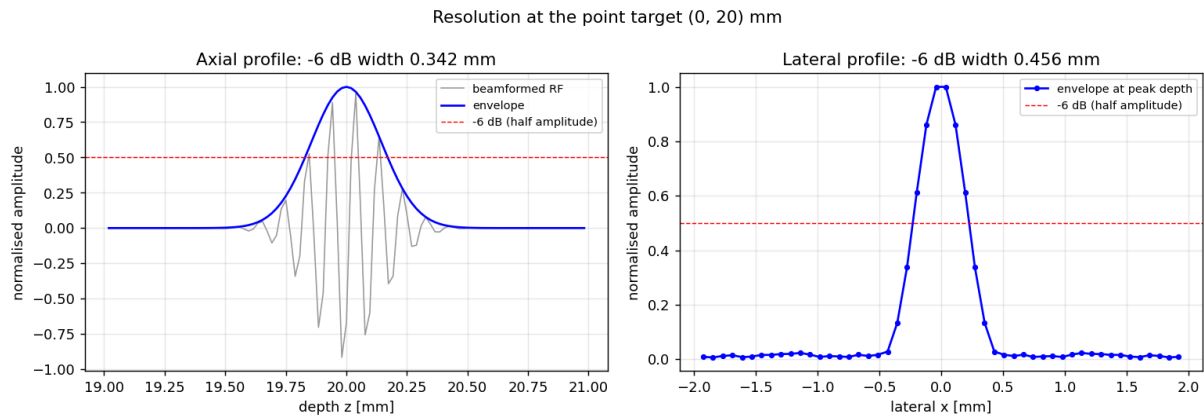


Display image of the cyst result (50 dB). The dark round cyst (dashed circle: true edge) is clear in the speckle.
Measured: the mean envelope inside $r < 4$ mm is 25.2 dB below the surrounding tissue.

5.4 Measured positions and resolution

True (x, z) mm	Measured (x, z) mm	Error x, z mm	Peak level
(-5, 10)	(-4.980, 10.010)	+0.020, +0.010	0.0 dB
(0, 20)	(-0.039, 20.001)	-0.039, +0.001	-5.4 dB
(+5, 30)	(+4.980, 29.992)	-0.020, -0.008	-11.5 dB

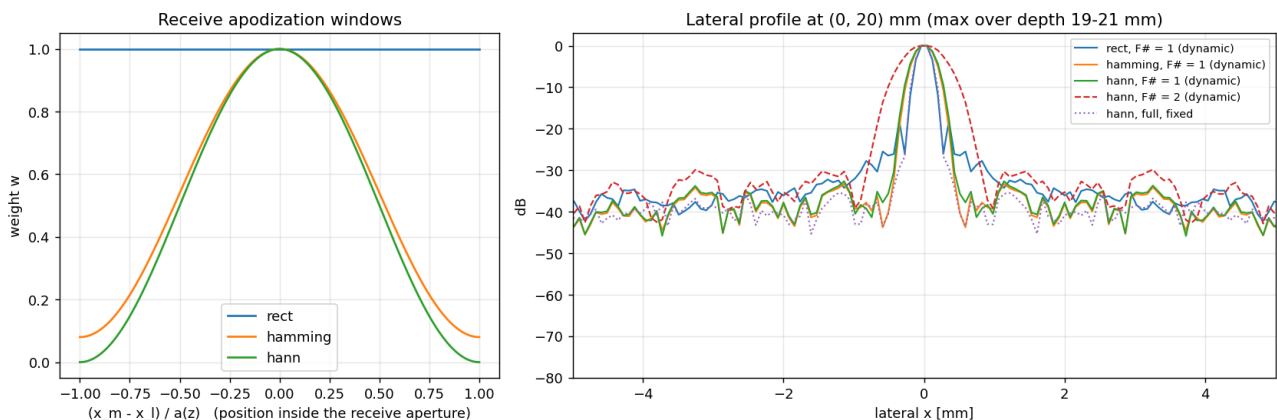
All positions are correct to within one grid step ($dz = 0.019$ mm, $dx = 0.078$ mm). There is no scan line exactly at $x = 0$ (256 lines from -10 to +10 mm), so the nearest line, $x = -0.039$ mm, is the best possible answer. Deeper targets are weaker because of attenuation.



Profiles through the (0, 20) mm target. Axial -6 dB width 0.342 mm; lateral -6 dB width 0.456 mm (Hann, $F\# = 1$).

5.5 Apodization and aperture comparison

Window	Aperture	Lateral -6 dB	Highest side lobe	Streak near 10 mm target
rect	$F\# = 1$	0.300 mm	-18.1 dB	-18.8 dB
hamming	$F\# = 1$	0.430 mm	-33.3 dB	-24.7 dB
hann (default)	$F\# = 1$	0.456 mm	-32.6 dB	-25.0 dB
hann	$F\# = 2$	0.816 mm	-29.8 dB	-26.2 dB
hann	full, fixed	0.315 mm	-35.5 dB	-17.7 dB



Left: the three receive windows. Right: lateral profile at the (0, 20) mm target (maximum over depth 19-21 mm) for each setting.

- Rect gives the narrowest spot (0.300 mm) but the highest side lobes (-18.1 dB). Hann and Hamming lower the side lobes to about -33 dB at the cost of a wider spot (0.43-0.46 mm). This is the usual trade-off.
- $F\# = 2$ uses half the aperture, so the spot is almost twice as wide (0.816 mm).
- The fixed full aperture gives a narrow spot at 20 mm, but the streak next to the shallow 10 mm target rises from -25.0 dB to -17.7 dB: at shallow depth the full aperture includes very steep angles, where the coarse pitch gives grating lobes. This is why $F\# = 1$ is used.
- The streak does not disappear with $F\# = 2$ (-26.2 dB), where the receive angles are too small for receive grating lobes. So most of the remaining streak is most likely already in the RF (it comes with the simple broadcast transmit) and is not caused by the receive delays.

5.6 Full console output

The complete console text of the run (also in output/console_log.txt; picture in output/console_screenshot.png). Times depend on the computer.

python3 run_team5.py

```
=====
NITK-UsoundSim | Team 5: Receive (RX) Beamforming, delay-and-sum
=====
Probe   : Philips L12-4 (FUS4103) reference model
          128 elements, pitch 0.30 mm, f0 8 MHz, fs 40 MHz, c 1540 m/s
Lines   : 256 scan lines, x = -10 .. +10 mm
Receive : apodization 'hann', F-number 1.0
Python  : 3.12.4 numpy 1.26.4
Beamformer loaded from: code/receive_beamforming.py

=====
STEP 1 Team 5 verification tests (code/verify_beamformer.py)
=====
Test RF is generated fresh by mock_data.py -> support/tissue_phantoms.simulate_rf ...

--- Test 1: Point-target focusing ---
    expected depth: 30.00 mm, measured peak: 29.99 mm (error 0.008 mm)
[PASS] peak within 0.096 mm tolerance

--- Test 2: Correct vs incorrect receive delay ---
    peak amplitude with correct delays: 0.327
    peak amplitude with no RX focusing: 0.066
[PASS] correct-delay peak is stronger than unfocused peak

--- Test 3: Lateral localization (two targets) ---
    strongest peak at x=-3.96 mm, z=24.99 mm
[PASS] strongest peak lands near one of the two targets

--- Test 4: Depth localization ---
    target z=15.0 mm -> measured peak z=15.03 mm (error 0.034 mm)
[PASS] depth error small at z=15.0 mm
    target z=40.0 mm -> measured peak z=40.04 mm (error 0.040 mm)
[PASS] depth error small at z=40.0 mm
    target z=55.0 mm -> measured peak z=55.04 mm (error 0.036 mm)
[PASS] depth error small at z=55.0 mm

Summary: 4/4 tests passed (6 PASS checks, 0 FAIL checks) [28.5 s]

=====
STEP 2 Load the sample inputs (input/*.npz)
=====
point_targets_rf.npz : 3 point targets at (-5,10), (0,20), (+5,30) mm
                      rf (128, 3574) (elements x time samples), |rf| max 1.71
                      t_axis -0.750 .. 88.575 us, step 25.0 ns (= 1/fs)
cyst_rf.npz          : speckle tissue with an anechoic cyst, centre (0,30) mm, radius 6 mm
                      rf (128, 3574) (elements x time samples), |rf| max 57.4
                      t_axis -0.750 .. 88.575 us, step 25.0 ns (= 1/fs)

    saved output/01_input_point_targets_raw_rf.png
    saved output/02_input_cyst_raw_rf.png

=====
STEP 3 The delay idea for the target at (0, 20) mm
=====
tau_TX at (0,20) mm = 12.987 us (nearest element fires first, broadcast)
tau_TX + tau_RX: centre element 25.975 us, edge element 1 30.923 us
-> the echo reaches the edge element 4.95 us later than the centre
peak |RF| on line x=0 near 20 mm: with RX delays 0.656, without per-element delays 0.175
    saved output/03_delay_alignment_0_20mm.png

=====
STEP 4 Beamform both inputs with das_beamform() (hann, F# = 1, 256 lines)
=====
point_targets : output (3118, 256) (depths x lines), z 0..60 mm step 0.0192 mm, 2.5 s
                saved output/beamformed_point_targets.npz (Team 5's output for the next stage)
cyst          : output (3118, 256) (depths x lines), z 0..60 mm step 0.0192 mm, 2.3 s
                saved output/beamformed_cyst.npz (Team 5's output for the next stage)
```

```

saved output/04_beamformed_rf_point_targets.png
saved output/05_display_point_targets.png
saved output/06_beamformed_rf_cyst.png
saved output/07_display_cyst.png

=====
STEP 5  Measurements on the beamformed point-target image
=====
(peaks are taken from the envelope of the beamformed RF;
grid step dz = 0.0192 mm, dx = 0.0784 mm)
  true (x, z) [mm] | measured (x, z) [mm] | error x, z [mm] | peak level
        (-5, 10) |      (-4.980, 10.010) |   +0.020, +0.010 |    0.0 dB
        (+0, 20) |      (-0.039, 20.001) |   -0.039, +0.001 |   -5.4 dB
        (+5, 30) |      (+4.980, 29.992) |   -0.020, -0.008 |  -11.5 dB

-6 dB resolution at the (0, 20) mm target (peak at x=-0.039, z=20.001 mm):
axial   width (along depth) : 0.342 mm
lateral width (across lines): 0.456 mm
(wavelength = 0.1925 mm; lateral line spacing 0.0784 mm)
saved output/08_resolution_profiles_0_20mm.png

Cyst image: mean envelope inside r < 4 mm of (0,30) mm = -25.2 dB
relative to the surrounding tissue ring (r = 7.5-10.5 mm)

=====
STEP 6  Apodization / aperture comparison at the (0, 20) mm target
=====
lateral width: -6 dB width across lines at the peak depth
highest side lobe: max over depth 19-21 mm of the envelope, outside the main lobe, |x| <= 5 mm
streak: brightest point right of the (-5,10) mm target (x > 1 mm, z 9.5-16 mm),
        in dB relative to the image maximum
  window      aperture | lateral -6 dB | highest side lobe | streak
  rect F# = 1 (dynamic) |    0.300 mm |    -18.1 dB | -18.8 dB
  hamming F# = 1 (dynamic) |    0.430 mm |    -33.3 dB | -24.7 dB
  hann F# = 1 (dynamic) |    0.456 mm |    -32.6 dB | -25.0 dB
  hann F# = 2 (dynamic) |    0.816 mm |    -29.8 dB | -26.2 dB
  hann      full, fixed |    0.315 mm |    -35.5 dB | -17.7 dB
saved output/09_apodization_comparison.png

=====
DONE
=====
Verification tests: 4/4 passed
Total run time: 46.6 s
All outputs are in: output/ (9 plots + console_screenshot.png, 2 beamformed .npz, console_log.txt)

```

```

o o o  python3 run_team5.py

=====
NITK-UsoundSim | Team 5: Receive (RX) Beamforming, delay-and-sum
=====
Probe   : Philips L12-4 (FU54103) reference model
         : 128 elements, pitch 0.30 mm, f0 8 MHz, fs 40 MHz, c 1540 m/s
Lines   : 256 scan lines, x = -10 .. +10 mm
Receive : apodization 'hann', F-number 1.0
Python  : 3.12.4  numpy 1.26.4
Beamformer loaded from: code/receive_beamforming.py

=====
STEP 1 Team 5 verification tests (code/verify_beamformer.py)
=====
Test RF is generated fresh by mock_data.py -> support/tissue_phantoms.simulate_rf ...

--- Test 1: Point-target focusing ---
    expected depth: 30.00 mm, measured peak: 29.99 mm (error 0.008 mm)
[PASS] peak within 0.096 mm tolerance

--- Test 2: Correct vs incorrect receive delay ---
    peak amplitude with correct delays: 0.327
    peak amplitude with no RX focusing: 0.066
[PASS] correct-delay peak is stronger than unfocused peak

--- Test 3: Lateral localization (two targets) ---
    strongest peak at x=-3.96 mm, z=24.99 mm
[PASS] strongest peak lands near one of the two targets

--- Test 4: Depth localization ---
    target z=15.0 mm -> measured peak z=15.03 mm (error 0.034 mm)
[PASS] depth error small at z=15.0 mm
    target z=40.0 mm -> measured peak z=40.04 mm (error 0.040 mm)
[PASS] depth error small at z=40.0 mm
    target z=55.0 mm -> measured peak z=55.04 mm (error 0.036 mm)
[PASS] depth error small at z=55.0 mm

Summary: 4/4 tests passed (6 PASS checks, 0 FAIL checks) [28.5 s]

=====
STEP 2 Load the sample inputs (input/*.npz)
=====
point_targets_rf.npz : 3 point targets at (-5,10), (0,20), (+5,30) mm
                      rf (128, 3574) (elements x time samples), |rf| max 1.71
                      t axis -0.750 .. 88.575 us, step 25.0 ns (= 1/fs)
cyst_rf.npz          : speckle tissue with an anechoic cyst, centre (0,30) mm, radius 6 mm
                      rf (128, 3574) (elements x time samples), |rf| max 57.4
                      t axis -0.750 .. 88.575 us, step 25.0 ns (= 1/fs)

    saved output/01_input_point_targets_raw_rf.png
    saved output/02_input_cyst_raw_rf.png

=====
STEP 3 The delay idea for the target at (0, 20) mm
=====
tau_TX at (0,20) mm = 12.987 us (nearest element fires first, broadcast)
tau_TX + tau_RX: centre element 25.975 us, edge element 130.923 us
-> the echo reaches the edge element 4.95 us later than the centre
peak |RF| on line x=0 near 20 mm: with RX delays 0.656, without per-element delays 0.175
    saved output/03_delay_alignment_0_20mm.png

=====
STEP 4 Beamform both inputs with das_beamform() (hann, F# = 1, 256 lines)
=====
point_targets : output (3118, 256) (depths x lines), z 0..60 mm step 0.0192 mm, 2.5 s
               saved output/beamformed_point_targets.npz (Team 5's output for the next stage)
cyst          : output (3118, 256) (depths x lines), z 0..60 mm step 0.0192 mm, 2.3 s
               saved output/beamformed_cyst.npz (Team 5's output for the next stage)
    saved output/04_beamformed_rf_point_targets.png
    saved output/05_display_point_targets.png
    saved output/06_beamformed_rf_cyst.png
    saved output/07_display_cyst.png

=====
STEP 5 Measurements on the beamformed point-target image
=====
(peaks are taken from the envelope of the beamformed RF;
grid step dz = 0.0192 mm, dx = 0.0784 mm)
   true (x, z) [mm] | measured (x, z) [mm] | error x, z [mm] | peak level
   (-5, 10) | (-4.980, 10.010) | +0.020, +0.010 | 0.0 dB
   (+0, 20) | (-0.039, 20.001) | -0.039, +0.001 | -5.4 dB
   (+5, 30) | (+4.980, 29.992) | -0.020, -0.008 | -11.5 dB

-6 dB resolution at the (0, 20) mm target (peak at x=-0.039, z=20.001 mm):
axial width (along depth) : 0.342 mm
lateral width (across lines): 0.456 mm
(wavelength = 0.1925 mm; lateral line spacing 0.0784 mm)
    saved output/08_resolution_profiles_0_20mm.png

Cyst image: mean envelope inside r < 4 mm of (0,30) mm = -25.2 dB
relative to the surrounding tissue ring (r = 7.5-10.5 mm)

=====
STEP 6 Apodization / aperture comparison at the (0, 20) mm target
=====
lateral width: -6 dB width across lines at the peak depth
highest side lobe: max over depth 19-21 mm of the envelope, outside the main lobe, |x| <= 5 mm
streak: brightest point right of the (-5,10) mm target (x > 1 mm, z 9.5-16 mm),
        in dB relative to the image maximum
   window  aperture | lateral -6 dB | highest side lobe | streak
   rect F# = 1 (dynamic) | 0.300 mm | -18.1 dB | -18.8 dB
   hamming F# = 1 (dynamic) | 0.430 mm | -33.3 dB | -24.7 dB
   hann F# = 1 (dynamic) | 0.456 mm | -32.6 dB | -25.0 dB
   hann F# = 2 (dynamic) | 0.816 mm | -29.8 dB | -26.2 dB
   hann full, fixed | 0.315 mm | -35.5 dB | -17.7 dB
    saved output/09_apodization_comparison.png

=====
DONE
=====
Verification tests: 4/4 passed
Total run time: 46.6 s
All outputs are in: output/ (9 plots + console_screenshot.png, 2 beamformed .npz, console_log.txt)

```

output/console_screenshot.png: the same console output as a picture.

5.7 List of output files

File	What it is
01_input_point_targets_raw_rf.png	raw RF input, point targets
02_input_cyst_raw_rf.png	raw RF input, cyst phantom
03_delay_alignment_0_20mm.png	echo curve before / after delays and the sum
04_beamformed_rf_point_targets.png	beamformed RF, point targets (Team 5 output)
05_display_point_targets.png	display image, point targets (display only)
06_beamformed_rf_cyst.png	beamformed RF, cyst (Team 5 output)
07_display_cyst.png	display image, cyst (display only)
08_resolution_profiles_0_20mm.png	axial and lateral -6 dB widths
09_apodization_comparison.png	rect / hamming / hann and aperture comparison
beamformed_point_targets.npz, beamformed_cyst.npz	beamformed RF (float32), z_axis, x_lines: data for the next stage
console_log.txt, console_screenshot.png	everything printed during the run

6 How to run

Step by step, on macOS, Linux or Windows

1. Install Python 3.9 or newer from python.org (on Windows tick 'Add Python to PATH').
2. Open a terminal (macOS: Terminal; Windows: Command Prompt or PowerShell) and go into this folder, e.g. `cd path/to/Team_5_RX_Beamforming`.
3. Install the three libraries once: `pip install -r requirements.txt` (macOS/Linux may need `pip3` or `python3 -m pip`).
4. Run: macOS / Linux `python3 run_team5.py`; Windows `python run_team5.py`.
5. Read the console, then open the output/ folder for the pictures and the log.

Expected runtime: about 30 to 40 seconds on a normal computer (measured 31.8 s and 36.2 s), up to about 1 minute when the computer is busy (measured up to 60 s). Most of the time is Team 5's tests, which simulate fresh RF.

Optional: to run only Team 5's tests, go into code/ and run `python3 verify_beamformer.py`.

Troubleshooting

Problem	What to do
ModuleNotFoundError: numpy / scipy / matplotlib	Run <code>pip install -r requirements.txt</code> with the same Python you use to run.
'python3' is not recognized (Windows)	Use <code>python run_team5.py</code> (or <code>py run_team5.py</code>).
FileNotFoundError for input/...npz	Run the command from inside the Team_5_RX_Beamforming folder and keep the folder structure.
The run seems slow	Normal on a busy computer; the tests take the longest. Wait until 'DONE'.
No window with plots opens	Correct: plots are saved as PNG files in output/, not shown on screen.
Permission error writing output/	Copy the folder to a place where you can write (e.g. Desktop).

7 Check yourself

Viva-style questions with short answers

Why does the echo of one point look like a curve across the elements?

Each element is at a different distance from the point, so the echo arrives at different times: first at the element above the point, later at the side elements (here 4.95 us later at the edge for the 20 mm target).

What are the two parts of the delay, and why is the transmit part a minimum?

τ_{TX} is the time for the transmit wave to reach the point, $\tau_{RX,m}$ the time back to element m . All elements fire together, so the wave arrives first from the nearest element: the minimum distance over n .

Why must the code use `np.interp(tau, t_axis, rf)` and not `rf[tau*fs]`?

The time axis starts at -0.75 us, not 0, so $\tau * fs$ would point to the wrong sample (about 0.58 mm too deep). `np.interp` uses the real times and also interpolates between samples.

What does the F-number do, and why $F\# = 1$ for this probe?

It sets the aperture size to $\text{depth}/F\#$, keeping the focus quality similar at all depths. $F\# = 1$ keeps receive angles below 26.6 deg, under the 39.9 deg grating-lobe angle of the coarse 0.30 mm pitch.

Rect vs Hann apodization: what changes?

Rect: narrower spot (0.300 mm) but higher side lobes (-18.1 dB). Hann: wider spot (0.456 mm) but much lower side lobes (-32.6 dB).

Is Team 5's output an image you can show directly?

No. It is beamformed RF (it still oscillates). The next stage takes the envelope and log compresses it; the display pictures here are only for checking.